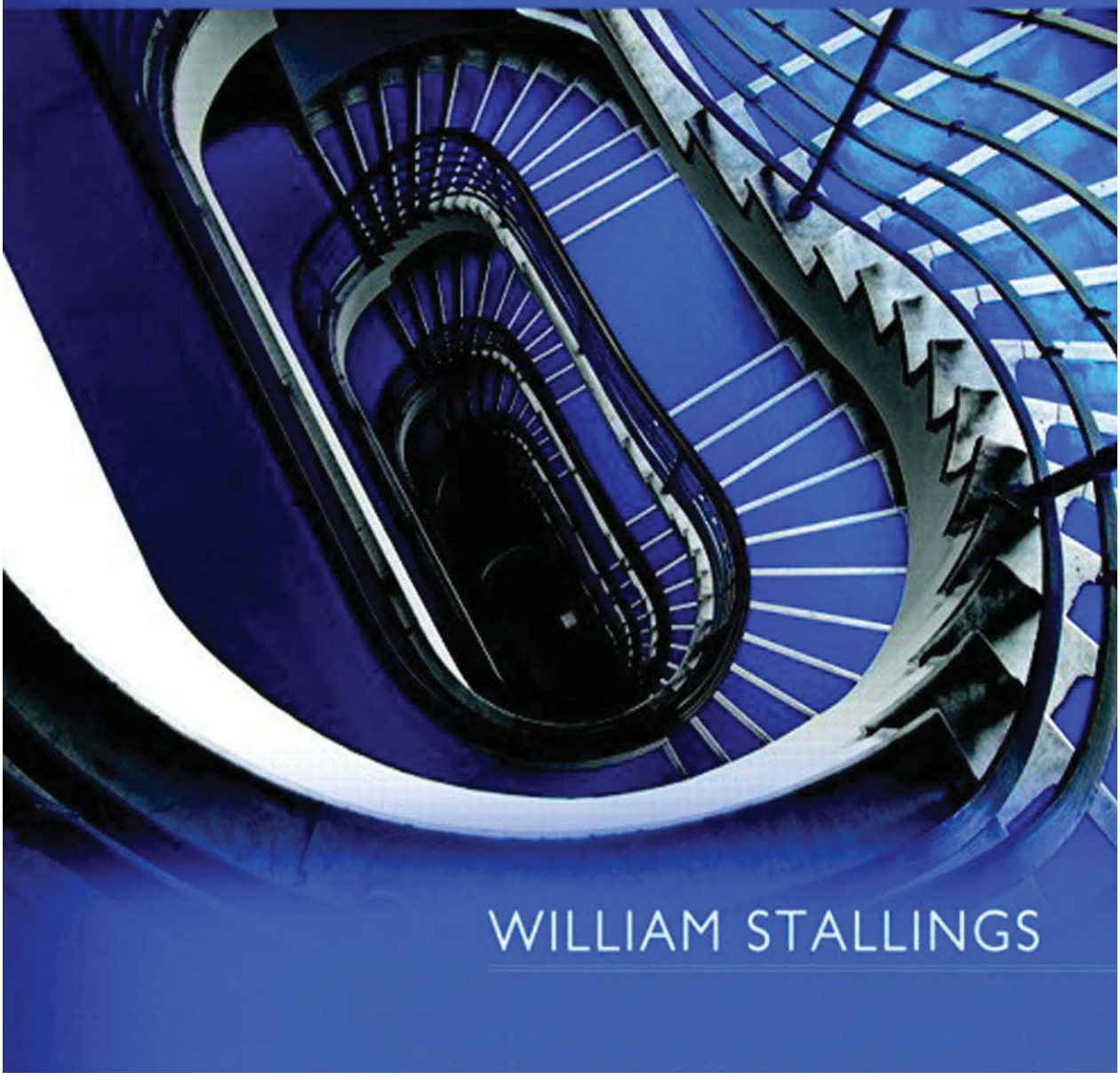
COMPUTER ORGANIZATION AND ARCHITECTURE

Designing for Performance

Tenth Edition



WILLIAM STALLINGS

COMPUTER ORGANIZATION AND ARCHITECTURE

DESIGNING FOR PERFORMANCE

TENTH EDITION

This page intentionally left blank

COMPUTER ORGANIZATION AND ARCHITECTURE
DESIGNING FOR PERFORMANCE
TENTH EDITION

William Stallings

With contribution by
Peter Zeno
University of Bridgeport

With Foreword by
Chris Jesshope
Professor (emeritus) University of Amsterdam

PEARSON

Boston • Columbus • Hoboken • Indianapolis • New York • San Francisco
Amsterdam • Cape Town • Dubai • London • Madrid • Milan • Munich • Paris • Montreal
Toronto • Delhi • Mexico City • São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei • Tokyo

Vice President and Editorial Director, ECS: *Marcia J. Horton*
Executive Editor: *Tracy Johnson (Dunkelberger)*
Editorial Assistant: *Kelsey Loanes*
Program Manager: *Carole Snyder*
Director of Product Management: *Erin Gregg*
Team Lead Product Management: *Scott Disanno*
Project Manager: *Robert Engelhardt*
Media Team Lead: *Steve Wright*
R&P Manager: *Rachel Youdelman*
R&P Senior Project Manager: *Timothy Nicholls*
Procurement Manager: *Mary Fischer*
Senior Specialist, Program Planning and Support: *Maura Zaldivar-Garcia*
Inventory Manager: *Bruce Boundy*
VP of Marketing: *Christy Lesko*
Director of Field Marketing: *Demetrius Hall*
Product Marketing Manager: *Bram van Kempen*
Marketing Assistant: *Jon Bryant*
Cover Designer: *Marta Samsel*
Cover Art: © *anderm / Fotolia*
Full-Service Project Management: *Mahalatchoumy Saravanan, Jouve India*
Printer/Binder: *Edwards Brothers Malloy*
Cover Printer: *Lehigh-Phoenix Color/Hagerstown*
Typeface: *Times Ten LT Std 10/12*

Copyright © 2016, 2013, 2010 Pearson Education, Inc., Hoboken, NJ 07030. All rights reserved. Manufactured in the United States of America. This publication is protected by Copyright and permissions should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. To obtain permission(s) to use materials from this work, please submit a written request to Pearson Higher Education, Permissions Department, 221 River Street, Hoboken, NJ 07030.

Many of the designations by manufacturers and seller to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed in initial caps or all caps. Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appears on page 833.

The author and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of theories and programs to determine their effectiveness. The author and publisher make no warranty of any kind, expressed or implied, with regard to these programs or the documentation contained in this book. The author and publisher shall not be liable in any event for incidental or consequential damages with, or arising out of, the furnishing, performance, or use of these programs.

Pearson Education Ltd., *London*
Pearson Education Australia Ply. Ltd., Sydney
Pearson Education Singapore, Pte. Ltd.
Pearson Education North Asia Ltd., *Hong Kong*
Pearson Education Canada, Inc., Toronto
Pearson Education de Mexico, S.A. de C.V.
Pearson Education–Japan, Tokyo
Pearson Education Malaysia, Pte. Ltd.
Pearson Education, Inc., *Hoboken, New Jersey*

Library of Congress Cataloging-in-Publication Data

Stallings, William.
Computer organization and architecture : designing for performance / William Stallings. — Tenth edition.
pages cm
Includes bibliographical references and index.
ISBN 978-0-13-410161-3 — ISBN 0-13-410161-8 1. Computer organization. 2. Computer architecture. I. Title.
QA76.9.C643S73 2016
004.2'2—dc23

2014044367

10 9 8 7 6 5 4 3 2 1

PEARSON

www.pearsonhighered.com

ISBN-10: 0-13-410161-8
ISBN-13: 978-0-13-410161-3

To Tricia
my loving wife, the kindest
and gentlest person

This page intentionally left blank

CONTENTS

Foreword xiii

Preface xv

About the Author xxiii

PART ONE INTRODUCTION 1

Chapter 1 Basic Concepts and Computer Evolution 1

1.1 Organization and Architecture 2
1.2 Structure and Function 3
1.3 A Brief History of Computers 11
1.4 The Evolution of the Intel x86 Architecture 27
1.5 Embedded Systems 29
1.6 Arm Architecture 33
1.7 Cloud Computing 39
1.8 Key Terms, Review Questions, and Problems 42

Chapter 2 Performance Issues 45

2.1 Designing for Performance 46
2.2 Multicore, Mics, and GPGPUs 52
2.3 Two Laws that Provide Insight: Ahmdahl's Law and Little's Law 53
2.4 Basic Measures of Computer Performance 56
2.5 Calculating the Mean 59
2.6 Benchmarks and Spec 67
2.7 Key Terms, Review Questions, and Problems 74

PART TWO THE COMPUTER SYSTEM 80

Chapter 3 A Top-Level View of Computer Function and Interconnection 80

3.1 Computer Components 81
3.2 Computer Function 83
3.3 Interconnection Structures 99
3.4 Bus Interconnection 100
3.5 Point-to-Point Interconnect 102
3.6 PCI Express 107
3.7 Key Terms, Review Questions, and Problems 116

Chapter 4 Cache Memory 120

4.1 Computer Memory System Overview 121
4.2 Cache Memory Principles 128
4.3 Elements of Cache Design 131
4.4 Pentium 4 Cache Organization 149
4.5 Key Terms, Review Questions, and Problems 152
Appendix 4A Performance Characteristics of Two-Level Memories 157

Chapter 5 Internal Memory 165

5.1 Semiconductor Main Memory 166
5.2 Error Correction 174
5.3 DDR DRAM 180
5.4 Flash Memory 185
5.5 Newer Nonvolatile Solid-State Memory Technologies 187
5.6 Key Terms, Review Questions, and Problems 190

Chapter 6 External Memory 194

6.1 Magnetic Disk 195
6.2 RAID 204
6.3 Solid State Drives 212
6.4 Optical Memory 217
6.5 Magnetic Tape 222
6.6 Key Terms, Review Questions, and Problems 224

Chapter 7 Input/Output 228

7.1 External Devices 230
7.2 I/O Modules 232
7.3 Programmed I/O 235
7.4 Interrupt-Driven I/O 239
7.5 Direct Memory Access 248
7.6 Direct Cache Access 254
7.7 I/O Channels and Processors 261
7.8 External Interconnection Standards 263
7.9 IBM zEnterprise EC12 I/O Structure 266
7.10 Key Terms, Review Questions, and Problems 270

Chapter 8 Operating System Support 275

8.1 Operating System Overview 276
8.2 Scheduling 287
8.3 Memory Management 293
8.4 Intel x86 Memory Management 304
8.5 Arm Memory Management 309
8.6 Key Terms, Review Questions, and Problems 314

PART THREE ARITHMETIC AND LOGIC 318

Chapter 9 Number Systems 318

9.1 The Decimal System 319
9.2 Positional Number Systems 320
9.3 The Binary System 321
9.4 Converting Between Binary and Decimal 321
9.5 Hexadecimal Notation 324
9.6 Key Terms and Problems 326

Chapter 10 Computer Arithmetic 328

10.1 The Arithmetic and Logic Unit 329
10.2 Integer Representation 330
10.3 Integer Arithmetic 335

10.4 Floating-Point Representation 350
10.5 Floating-Point Arithmetic 358
10.6 Key Terms, Review Questions, and Problems 367

Chapter 11 Digital Logic 372

11.1 Boolean Algebra 373
11.2 Gates 376
11.3 Combinational Circuits 378
11.4 Sequential Circuits 396
11.5 Programmable Logic Devices 405
11.6 Key Terms and Problems 409

PART FOUR THE CENTRAL PROCESSING UNIT 412

Chapter 12 Instruction Sets: Characteristics and Functions 412

12.1 Machine Instruction Characteristics 413
12.2 Types of Operands 420
12.3 Intel x86 and ARM Data Types 422
12.4 Types of Operations 425
12.5 Intel x86 and ARM Operation Types 438
12.6 Key Terms, Review Questions, and Problems 446
Appendix 12A Little-, Big-, and Bi-Endian 452

Chapter 13 Instruction Sets: Addressing Modes and Formats 456

13.1 Addressing Modes 457
13.2 x86 and ARM Addressing Modes 463
13.3 Instruction Formats 469
13.4 x86 and ARM Instruction Formats 477
13.5 Assembly Language 482
13.6 Key Terms, Review Questions, and Problems 484

Chapter 14 Processor Structure and Function 488

14.1 Processor Organization 489
14.2 Register Organization 491
14.3 Instruction Cycle 496
14.4 Instruction Pipelining 500
14.5 The x86 Processor Family 517
14.6 The ARM Processor 524
14.7 Key Terms, Review Questions, and Problems 530

Chapter 15 Reduced Instruction Set Computers 535

15.1 Instruction Execution Characteristics 537
15.2 The Use of a Large Register File 542
15.3 Compiler-Based Register Optimization 547
15.4 Reduced Instruction Set Architecture 549
15.5 RISC Pipelining 555
15.6 MIPS R4000 559
15.7 SPARC 565
15.8 RISC versus CISC Controversy 570
15.9 Key Terms, Review Questions, and Problems 571

Chapter 16 Instruction-Level Parallelism and Superscalar Processors 575

16.1 Overview 576
16.2 Design Issues 581
16.3 Intel Core Microarchitecture 591
16.4 ARM Cortex-A8 596
16.5 ARM Cortex-M3 604
16.6 Key Terms, Review Questions, and Problems 608

PART FIVE PARALLEL ORGANIZATION 613

Chapter 17 Parallel Processing 613

17.1 Multiple Processor Organizations 615
17.2 Symmetric Multiprocessors 617
17.3 Cache Coherence and the MESI Protocol 621
17.4 Multithreading and Chip Multiprocessors 628
17.5 Clusters 633
17.6 Nonuniform Memory Access 640
17.7 Cloud Computing 643
17.8 Key Terms, Review Questions, and Problems 650

Chapter 18 Multicore Computers 656

18.1 Hardware Performance Issues 657
18.2 Software Performance Issues 660
18.3 Multicore Organization 665
18.4 Heterogeneous Multicore Organization 667
18.5 Intel Core i7-990X 676
18.6 ARM Cortex-A15 MPCore 677
18.7 IBM zEnterprise EC12 Mainframe 682
18.8 Key Terms, Review Questions, and Problems 685

Chapter 19 General-Purpose Graphic Processing Units 688

19.1 Cuda Basics 689
19.2 GPU versus CPU 691
19.3 GPU Architecture Overview 692
19.4 Intel's Gen8 GPU 701
19.5 When to Use a GPU as a Coprocessor 704
19.6 Key Terms and Review Questions 706

PART SIX THE CONTROL UNIT 707

Chapter 20 Control Unit Operation 707

20.1 Micro-Operations 708
20.2 Control of the Processor 714
20.3 Hardwired Implementation 724
20.4 Key Terms, Review Questions, and Problems 727

Chapter 21 Microprogrammed Control 729

21.1 Basic Concepts 730
21.2 Microinstruction Sequencing 739

21.3 Microinstruction Execution 745
21.4 TI 8800 755
21.5 Key Terms, Review Questions, and Problems 766

Appendix A Projects for Teaching Computer Organization and Architecture 768

A.1 Interactive Simulations 769
A.2 Research Projects 771
A.3 Simulation Projects 771
A.4 Assembly Language Projects 772
A.5 Reading/Report Assignments 773
A.6 Writing Assignments 773
A.7 Test Bank 773

Appendix B Assembly Language and Related Topics 774

B.1 Assembly Language 775
B.2 Assemblers 783
B.3 Loading and Linking 787
B.4 Key Terms, Review Questions, and Problems 795

References 800

Index 809

Credits 833

ONLINE APPENDICES[1]

Appendix C System Buses
Appendix D Protocols and Protocol Architectures
Appendix E Scrambling
Appendix F Victim Cache Strategies
Appendix G Interleaved Memory
Appendix H International Reference Alphabet
Appendix I Stacks
Appendix J Thunderbolt and Infiniband
Appendix K Virtual Memory Page Replacement Algorithms
Appendix L Hash Tables
Appendix M Recursive Procedures
Appendix N Additional Instruction Pipeline Topics
Appendix O Timing Diagrams
Glossary

[1]Online chapters, appendices, and other documents are Premium Content, available via the access card at the front of this book.

This page intentionally left blank



FOREWORD

by Chris Jesshope
Professor (emeritus) University of Amsterdam
Author of Parallel Computers (with R W Hockney), 1981 & 1988

Having been active in computer organization and architecture for many years, it is a pleasure to write this foreword for the new edition of William Stallings' comprehensive book on this subject. In doing this, I found myself reflecting on the trends and changes in this subject over the time that I have been involved in it. I myself became interested in computer architecture at a time of significant innovation and disruption. That disruption was brought about not only through advances in technology but perhaps more significantly through access to that technology. VLSI was here and VLSI design was available to students in the classroom. These were exciting times. The ability to integrate a mainframe style computer on a single silicon chip was a milestone, but that this was accomplished by an academic research team made the achievement quite unique. This period was characterized by innovation and diversity in computer architecture with one of the main trends being in the area of parallelism. In the 1970s, I had hands-on experience of the Illiac IV, which was an early example of explicit parallelism in computer architecture and which incidentally pioneered all semiconductor memory. This interaction, and it certainly was that, kick-started my own interest in computer architecture and organization, with particular emphasis on explicit parallelism in computer architecture.

Throughout the 1980s and early 1990s research flourished in this field and there was a great deal of innovation, much of which came to market through university start-ups. Ironically however, it was the same technology that reversed this trend. Diversity was gradually replaced with a near monoculture in computer systems with advances in just a few instruction set architectures. Moore's law, a self-fulfilling prediction that became an industry guideline, meant that basic device speeds and integration densities both grew exponentially, with the latter doubling every 18 months of so. The speed increase was the proverbial free lunch for computer architects and the integration levels allowed more complexity and innovation at the micro-architecture level. The free lunch of course did have a cost, that being the exponential growth of capital investment required to fulfill Moore's law, which once again limited the access to state-of-the-art technologies. Moreover, most users found it easier to wait for the next generation of mainstream processor than to invest in the innovations in parallel computers, with their pitfalls and difficulties. The exceptions to this were the few large institutions requiring ultimate performance; two topical examples being large-scale scientific simulation such as climate modeling and also in our security services for code breaking. For

everyone else, the name of the game was compatibility and two instruction set architectures that benefited from this were x86 and ARM, the latter in embedded systems and the former in just about everything else. Parallelism was still there in the implementation of these ISAs, it was just that it was implicit, harnessed by the architecture not in the instruction stream that drives it.

Throughout the late 1990s and early 2000s, this approach to implicitly exploiting concurrency in single-core computer systems flourished. However, in spite of the exponential growth of logic density, it was the cost of the techniques exploited which brought this era to a close. In superscalar processors, the logic costs do not grow linearly with issue width (parallelism), while some components grow as the square or even the cube of the issue width. Although the exponential growth in logic could sustain this continued development, there were two major pitfalls: it was increasingly difficult to expose concurrency implicitly from imperative programs and hence efficiencies in the use of instruction issue slots decreased. Perhaps more importantly, technology was experiencing a new barrier to performance gains, namely that of power dissipation, and several superscalar developments were halted because the silicon in them would have been too hot. These constraints have mandated the exploitation of explicit parallelism, despite the compatibility challenges. So it seems that again innovation and diversity are opening up this area to new research.

Perhaps not since the 1980s has it been so interesting to study in this field. That diversity is an economic reality can be seen by the decrease in issue width (implicit parallelism) and increase in the number of cores (explicit parallelism) in mainstream processors. However, the question is how to exploit this, both at the application and the system level. There are significant challenges here still to be solved. Superscalar processors rely on the processor to extract parallelism from a single instruction stream. What if we shifted the emphasis and provided an instruction stream with maximum parallelism, how can we exploit this in different configurations and/or generations of processors that require different levels of explicit parallelism? Is it possible therefore to have a micro-architecture that sequentializes and schedules this maximum concurrency captured in the ISA to match the current configuration of cores so that we gain the same compatibility in a world of explicit parallelism? Does this require operating systems in silicon for efficiency?

These are just some of the questions facing us today. To answer these questions and more requires a sound foundation in computer organization and architecture, and this book by William Stallings provides a very timely and comprehensive foundation. It gives a complete introduction to the basics required, tackling what can be quite complex topics with apparent simplicity. Moreover, it deals with the more recent developments in this field, where innovation has in the past, and is, currently taking place. Examples are in superscalar issue and in explicitly parallel multicores. What is more, this latest edition includes two very recent topics in the design and use of GPUs for general-purpose use and the latest trends in cloud computing, both of which have become mainstream only recently. The book makes good use of examples throughout to highlight the theoretical issues covered, and most of these examples are drawn from developments in the two most widely used ISAs, namely the x86 and ARM. To reiterate, this book is complete and is a pleasure to read and hopefully will kick-start more young researchers down the same path that I have enjoyed over the last 40 years!



PREFACE

WHAT'S NEW IN THE TENTH EDITION

Since the ninth edition of this book was published, the field has seen continued innovations and improvements. In this new edition, I try to capture these changes while maintaining a broad and comprehensive coverage of the entire field. To begin this process of revision, the ninth edition of this book was extensively reviewed by a number of professors who teach the subject and by professionals working in the field. The result is that, in many places, the narrative has been clarified and tightened, and illustrations have been improved.

Beyond these refinements to improve pedagogy and user-friendliness, there have been substantive changes throughout the book. Roughly the same chapter organization has been retained, but much of the material has been revised and new material has been added. The most noteworthy changes are as follows:

- **GPGPU [General-Purpose Computing on Graphics Processing Units (GPUs)]:** One of the most important new developments in recent years has been the broad adoption of GPGPUs to work in coordination with traditional CPUs to handle a wide range of applications involving large arrays of data. A new chapter is devoted to the topic of GPGPUs.
- **Heterogeneous multicore processors:** The latest development in multicore architecture is the heterogeneous multicore processor. A new section in the chapter on multicore processors surveys the various types of heterogeneous multicore processors.
- **Embedded systems:** The overview of embedded systems in Chapter 1 has been substantially revised and expanded to reflect the current state of embedded technology.
- **Microcontrollers:** In terms of numbers, almost all computers now in use are embedded microcontrollers. The treatment of embedded systems in Chapter 1 now includes coverage of microcontrollers. The ARM Cortex-M3 microcontroller is used as an example system throughout the text.
- **Cloud computing:** New to this edition is a discussion of cloud computing, with an overview in Chapter 1 and more detailed treatment in Chapter 17.
- **System performance:** The coverage of system performance issues has been revised, expanded, and reorganized for a clearer and more thorough treatment. Chapter 2 is devoted to this topic, and the issue of system performance arises throughout the book.

- **Flash memory:** The coverage of flash memory has been updated and expanded, and now includes a discussion of the technology and organization of flash memory for internal memory (Chapter 5) and external memory (Chapter 6).
- **Nonvolatile RAM:** New to this edition is treatment of three important new nonvolatile solid-state RAM technologies that occupy different positions in the memory hierarchy: STT-RAM, PCRAM, and ReRAM.
- **Direct cache access (DCA):** To meet the protocol processing demands for very high speed network connections, Intel and other manufacturers have developed DCA technologies that provide much greater throughput than traditional direct memory access (DMA) approaches. New to this edition, Chapter 7 explores DCA in some detail.
- **Intel Core Microarchitecture:** As in the previous edition, the Intel x86 family is used as a major example system throughout. The treatment has been updated to reflect newer Intel systems, especially the Intel Core Microarchitecture, which is used on both PC and server products.
- **Homework problems:** The number of supplemental homework problems, with solutions, available for student practice has been expanded.

SUPPORT OF ACM/IEEE COMPUTER SCIENCE CURRICULA 2013

The book is intended for both an academic and a professional audience. As a textbook, it is intended as a one- or two-semester undergraduate course for computer science, computer engineering, and electrical engineering majors. This edition is designed to support the recommendations of the ACM/IEEE Computer Science Curricula 2013 (CS2013). CS2013 divides all course work into three categories: Core-Tier 1 (all topics should be included in the curriculum); Core-Tier-2 (all or almost all topics should be included); and Elective (desirable to provide breadth and depth). In the Architecture and Organization (AR) area, CS2013 includes five Tier-2 topics and three Elective topics, each of which has a number of subtopics. This text covers all eight topics listed by CS2013. Table P.1 shows the support for the AR Knowledge Area provided in this textbook.

Table P.1 Coverage of CS2013 Architecture and Organization (AR) Knowledge Area

IAS Knowledge Units	Topics	Textbook Coverage
Digital Logic and Digital Systems (Tier 2)	• Overview and history of computer architecture • Combinational vs. sequential logic/Field programmable gate arrays as a fundamental combinational sequential logic building block • Multiple representations/layers of interpretation (hardware is just another layer) • Physical constraints (gate delays, fan-in, fan-out, energy/power)	—Chapter 1 —Chapter 11
Machine Level Representation of Data (Tier 2)	• Bits, bytes, and words • Numeric data representation and number bases • Fixed- and floating-point systems • Signed and twos-complement representations • Representation of non-numeric data (character codes, graphical data)	—Chapter 9 —Chapter 10

IAS Knowledge Units	Topics	Textbook Coverage
Assembly Level Machine Organization (Tier 2)	• Basic organization of the von Neumann machine • Control unit; instruction fetch, decode, and execution • Instruction sets and types (data manipulation, control, I/O) • Assembly/machine language programming • Instruction formats • Addressing modes • Subroutine call and return mechanisms (cross-reference PL/Language Translation and Execution) • I/O and interrupts • Shared memory multiprocessors/multicore organization • Introduction to SIMD vs. MIMD and the Flynn Taxonomy	—Chapter 1 —Chapter 7 —Chapter 12 —Chapter 13 —Chapter 17 —Chapter 18 —Chapter 20 —Chapter 21 —Appendix A
Memory System Organization and Architecture (Tier 2)	• Storage systems and their technology • Memory hierarchy: temporal and spatial locality • Main memory organization and operations • Latency, cycle time, bandwidth, and interleaving • Cache memories (address mapping, block size, replacement and store policy) • Multiprocessor cache consistency/Using the memory system for inter-core synchronization/atomic memory operations • Virtual memory (page table, TLB) • Fault handling and reliability	—Chapter 4 —Chapter 5 —Chapter 6 —Chapter 8 —Chapter 17
Interfacing and Communication (Tier 2)	• I/O fundamentals: handshaking, buffering, programmed I/O, interrupt-driven I/O • Interrupt structures: vectored and prioritized, interrupt acknowledgment • External storage, physical organization, and drives • Buses: bus protocols, arbitration, direct-memory access (DMA) • RAID architectures	—Chapter 3 —Chapter 6 —Chapter 7
Functional Organization (Elective)	• Implementation of simple datapaths, including instruction pipelining, hazard detection, and resolution • Control unit: hardwired realization vs. microprogrammed realization • Instruction pipelining • Introduction to instruction-level parallelism (ILP)	—Chapter 14 —Chapter 16 —Chapter 20 —Chapter 21
Multiprocessing and Alternative Architectures (Elective)	• Example SIMD and MIMD instruction sets and architectures • Interconnection networks • Shared multiprocessor memory systems and memory consistency • Multiprocessor cache coherence	—Chapter 12 —Chapter 13 —Chapter 17
Performance Enhancements (Elective)	• Superscalar architecture • Branch prediction, Speculative execution, Out-of-order execution • Prefetching • Vector processors and GPUs • Hardware support for multithreading • Scalability	—Chapter 15 —Chapter 16 —Chapter 19

OBJECTIVES

This book is about the structure and function of computers. Its purpose is to present, as clearly and completely as possible, the nature and characteristics of modern-day computer systems.

This task is challenging for several reasons. First, there is a tremendous variety of products that can rightly claim the name of computer, from single-chip microprocessors costing a few dollars to supercomputers costing tens of millions of dollars. Variety is exhibited not only in cost but also in size, performance, and application. Second, the rapid pace of change that has always characterized computer technology continues with no letup. These changes cover all aspects of computer technology, from the underlying integrated circuit technology used to construct computer components to the increasing use of parallel organization concepts in combining those components.

In spite of the variety and pace of change in the computer field, certain fundamental concepts apply consistently throughout. The application of these concepts depends on the current state of the technology and the price/performance objectives of the designer. The intent of this book is to provide a thorough discussion of the fundamentals of computer organization and architecture and to relate these to contemporary design issues.

The subtitle suggests the theme and the approach taken in this book. It has always been important to design computer systems to achieve high performance, but never has this requirement been stronger or more difficult to satisfy than today. All of the basic performance characteristics of computer systems, including processor speed, memory speed, memory capacity, and interconnection data rates, are increasing rapidly. Moreover, they are increasing at different rates. This makes it difficult to design a balanced system that maximizes the performance and utilization of all elements. Thus, computer design increasingly becomes a game of changing the structure or function in one area to compensate for a performance mismatch in another area. We will see this game played out in numerous design decisions throughout the book.

A computer system, like any system, consists of an interrelated set of components. The system is best characterized in terms of structure—the way in which components are interconnected, and function—the operation of the individual components. Furthermore, a computer's organization is hierarchical. Each major component can be further described by decomposing it into its major subcomponents and describing their structure and function. For clarity and ease of understanding, this hierarchical organization is described in this book from the top down:

- **Computer system:** Major components are processor, memory, I/O.
- **Processor:** Major components are control unit, registers, ALU, and instruction execution unit.
- **Control unit:** Provides control signals for the operation and coordination of all processor components. Traditionally, a microprogramming implementation has been used, in which major components are control memory, microinstruction sequencing logic, and registers. More recently, microprogramming has been less prominent but remains an important implementation technique.

The objective is to present the material in a fashion that keeps new material in a clear context. This should minimize the chance that the reader will get lost and should provide better motivation than a bottom-up approach.

Throughout the discussion, aspects of the system are viewed from the points of view of both architecture (those attributes of a system visible to a machine language programmer) and organization (the operational units and their interconnections that realize the architecture).

EXAMPLE SYSTEMS

This text is intended to acquaint the reader with the design principles and implementation issues of contemporary operating systems. Accordingly, a purely conceptual or theoretical treatment would be inadequate. To illustrate the concepts and to tie them to real-world design choices that must be made, two processor families have been chosen as running examples:

- **Intel x86 architecture:** The x86 architecture is the most widely used for nonembedded computer systems. The x86 is essentially a complex instruction set computer (CISC) with some RISC features. Recent members of the x86 family make use of superscalar and multicore design principles. The evolution of features in the x86 architecture provides a unique case-study of the evolution of most of the design principles in computer architecture.
- **ARM:** The ARM architecture is arguably the most widely used embedded processor, used in cell phones, iPods, remote sensor equipment, and many other devices. The ARM is essentially a reduced instruction set computer (RISC). Recent members of the ARM family make use of superscalar and multicore design principles.

Many, but by no means all, of the examples in this book are drawn from these two computer families. Numerous other systems, both contemporary and historical, provide examples of important computer architecture design features.

PLAN OF THE TEXT

The book is organized into six parts:

- Overview
- The computer system
- Arithmetic and logic
- The central processing unit
- Parallel organization, including multicore
- The control unit

The book includes a number of pedagogic features, including the use of interactive simulations and numerous figures and tables to clarify the discussion. Each chapter includes a list of key words, review questions, homework problems, and suggestions for further reading. The book also includes an extensive glossary, a list of frequently used acronyms, and a bibliography.

INSTRUCTOR SUPPORT MATERIALS

Support materials for instructors are available at the **Instructor Resource Center (IRC)** for this textbook, which can be reached through the publisher's Web site www.pearsonhighered.com/stallings or by clicking on the link labeled "Pearson Resources for Instructors" at this

book's Companion Web site at WilliamStallings.com/ComputerOrganization. To gain access to the IRC, please contact your local Pearson sales representative via pearsonhighered.com/educator/replocator/requestSalesRep.page or call Pearson Faculty Services at 1-800-526-0485. The IRC provides the following materials:

- **Projects manual:** Project resources including documents and portable software, plus suggested project assignments for all of the project categories listed subsequently in this Preface.
- **Solutions manual:** Solutions to end-of-chapter Review Questions and Problems.
- **PowerPoint slides:** A set of slides covering all chapters, suitable for use in lecturing.
- **PDF files:** Copies of all figures and tables from the book.
- **Test bank:** A chapter-by-chapter set of questions.
- **Sample syllabuses:** The text contains more material than can be conveniently covered in one semester. Accordingly, instructors are provided with several sample syllabuses that guide the use of the text within limited time. These samples are based on real-world experience by professors with the first edition.

The **Companion Web site**, at WilliamStallings.com/ComputerOrganization (click on Instructor Resources link) includes the following:

- Links to Web sites for other courses being taught using this book.
- Sign-up information for an Internet mailing list for instructors using this book to exchange information, suggestions, and questions with each other and with the author.

STUDENT RESOURCES



For this new edition, a tremendous amount of original supporting material for students has been made available online, at two Web locations. The **Companion Web Site**, at WilliamStallings.com/ComputerOrganization (click on Student Resources link), includes a list of relevant links organized by chapter and an errata sheet for the book.

Purchasing this textbook new grants the reader six months of access to the **Premium Content Site**, which includes the following materials:

- **Online chapters:** To limit the size and cost of the book, two chapters of the book are provided in PDF format. The chapters are listed in this book's table of contents.
- **Online appendices:** There are numerous interesting topics that support material found in the text but whose inclusion is not warranted in the printed text. A total of 13 appendices cover these topics for the interested student. The appendices are listed in this book's table of contents.
- **Homework problems and solutions:** To aid the student in understanding the material, a separate set of homework problems with solutions are available. Students can enhance their understanding of the material by working out the solutions to these problems and then checking their answers.



To access the Premium Content site, click on the *Premium Content* link at the Companion Web site or at pearsonhighered.com/stallings and enter the student access code found on the card in the front of the book.

Finally, I maintain the Computer Science Student Resource Site at **WilliamStallings.com/StudentSupport.html**.

PROJECTS AND OTHER STUDENT EXERCISES

For many instructors, an important component of a computer organization and architecture course is a project or set of projects by which the student gets hands-on experience to reinforce concepts from the text. This book provides an unparalleled degree of support for including a projects component in the course. The instructor's support materials available through Prentice Hall not only includes guidance on how to assign and structure the projects but also includes a set of user's manuals for various project types plus specific assignments, all written especially for this book. Instructors can assign work in the following areas:

- **Interactive simulation assignments:** Described subsequently.
- **Research projects:** A series of research assignments that instruct the student to research a particular topic on the Internet and write a report.
- **Simulation projects:** The IRC provides support for the use of the two simulation packages: SimpleScalar can be used to explore computer organization and architecture design issues. SMPCache provides a powerful educational tool for examining cache design issues for symmetric multiprocessors.
- **Assembly language projects:** A simplified assembly language, CodeBlue, is used and assignments based on the popular Core Wars concept are provided.
- **Reading/report assignments:** A list of papers in the literature, one or more for each chapter, that can be assigned for the student to read and then write a short report.
- **Writing assignments:** A list of writing assignments to facilitate learning the material.
- **Test bank:** Includes T/F, multiple choice, and fill-in-the-blank questions and answers.

This diverse set of projects and other student exercises enables the instructor to use the book as one component in a rich and varied learning experience and to tailor a course plan to meet the specific needs of the instructor and students. See Appendix A in this book for details.

INTERACTIVE SIMULATIONS

An important feature in this edition is the incorporation of interactive simulations. These simulations provide a powerful tool for understanding the complex design features of a modern computer system. A total of 20 interactive simulations are used to illustrate key functions and algorithms in computer organization and architecture design. At the relevant point in the book, an icon indicates that a relevant interactive simulation is available online for student use. Because the animations enable the user to set initial conditions, they can

serve as the basis for student assignments. The instructor's supplement includes a set of assignments, one for each of the animations. Each assignment includes several specific problems that can be assigned to students.

For access to the animations, click on the rotating globe at this book's Web site at http://williamstallings.com/ComputerOrganization.

ACKNOWLEDGMENTS

This new edition has benefited from review by a number of people, who gave generously of their time and expertise. The following professors and instructors reviewed all or a large part of the manuscript: Molisa Derk (Dickinson State University), Yaohang Li (Old Dominion University), Dwayne Ockel (Regis University), Nelson Luiz Passos (Midwestern State University), Mohammad Abdus Salam (Southern University), and Vladimir Zwass (Fairleigh Dickinson University).

Thanks also to the many people who provided detailed technical reviews of one or more chapters: Rekai Gonzalez Alberquilla, Allen Baum, Jalil Boukhobza, Dmitry Bufistov, Humberto Calderón, Jesus Carretero, Ashkan Eghbal, Peter Glaskowsky, Ram Huggahalli, Chris Jesshope, Athanasios Kakarountas, Isil Oz, Mitchell Poplingher, Roger Shepherd, Jigar Savla, Karl Stevens, Siri Uppalapati, Dr. Sriram Vajapeyam, Kugan Vivekanandarajah, Pooria M. Yaghini, and Peter Zeno,

Peter Zeno also contributed Chapter 19 on GPGPUs.

Professor Cindy Norris of Appalachian State University, Professor Bin Mu of the University of New Brunswick, and Professor Kenrick Mock of the University of Alaska kindly supplied homework problems.

Aswin Sreedhar of the University of Massachusetts developed the interactive simulation assignments and also wrote the test bank.

Professor Miguel Angel Vega Rodriguez, Professor Dr. Juan Manuel Sánchez Pérez, and Professor Dr. Juan Antonio Gómez Pulido, all of University of Extremadura, Spain, prepared the SMPCache problems in the instructor's manual and authored the SMPCache User's Guide.

Todd Bezenek of the University of Wisconsin and James Stine of Lehigh University prepared the SimpleScalar problems in the instructor's manual, and Todd also authored the SimpleScalar User's Guide.

Finally, I would like to thank the many people responsible for the publication of the book, all of whom did their usual excellent job. This includes the staff at Pearson, particularly my editor Tracy Johnson, her assistant Kelsey Loanes, program manager Carole Snyder, and production manager Bob Engelhardt. I also thank Mahalatchoumy Saravanan and the production staff at Jouve India for another excellent and rapid job. Thanks also to the marketing and sales staffs at Pearson, without whose efforts this book would not be in front of you.

About the Author



Dr. William Stallings has authored 17 textbooks, and counting revised editions, over 40 books on computer security, computer networking, and computer architecture. In over 30 years in the field, he has been a technical contributor, technical manager, and an executive with several high-technology firms. Currently, he is an independent consultant whose clients have included computer and networking manufacturers and customers, software development firms, and leading-edge government research institutions. He has 13 times received the award for the best computer science textbook of the year from the Text and Academic Authors Association.

He created and maintains the Computer Science Student Resource Site at ComputerScienceStudent.com. This site provides documents and links on a variety of subjects of general interest to computer science students (and professionals). His articles appear regularly at networking.answers.com, where he is the Networking Category Expert Writer. He is a member of the editorial board of *Cryptologia*, a scholarly journal devoted to all aspects of cryptology.

Dr. Stallings holds a PhD from MIT in computer science and a BS from Notre Dame in electrical engineering.

This page intentionally left blank



PART ONE INTRODUCTION

CHAPTER 1

BASIC CONCEPTS AND COMPUTER EVOLUTION

1.1 **Organization and Architecture**

1.2 **Structure and Function**
- Function
- Structure

1.3 **A Brief History of Computers**
- The First Generation: Vacuum Tubes
- The Second Generation: Transistors
- The Third Generation: Integrated Circuits
- Later Generations

1.4 **The Evolution of the Intel x86 Architecture**

1.5 **Embedded Systems**
- The Internet of Things
- Embedded Operating Systems
- Application Processors versus Dedicated Processors
- Microprocessors versus Microcontrollers
- Embedded versus Deeply Embedded Systems

1.6 **ARM Architecture**
- ARM Evolution
- Instruction Set Architecture
- ARM Products

1.7 **Cloud Computing**
- Basic Concepts
- Cloud Services

1.8 **Key Terms, Review Questions, and Problems**

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- Explain the general functions and structure of a digital computer.
- Present an overview of the evolution of computer technology from early digital computers to the latest microprocessors.
- Present an overview of the evolution of the x86 architecture.
- Define embedded systems and list some of the requirements and constraints that various embedded systems must meet.

1.1 ORGANIZATION AND ARCHITECTURE

In describing computers, a distinction is often made between *computer architecture* and *computer organization*. Although it is difficult to give precise definitions for these terms, a consensus exists about the general areas covered by each. For example, see [VRAN80], [SIEW82], and [BELL78a]; an interesting alternative view is presented in [REDD76].

Computer architecture refers to those attributes of a system visible to a programmer or, put another way, those attributes that have a direct impact on the logical execution of a program. A term that is often used interchangeably with computer architecture is **instruction set architecture (ISA)**. The ISA defines instruction formats, instruction opcodes, registers, instruction and data memory; the effect of executed instructions on the registers and memory; and an algorithm for controlling instruction execution. **Computer organization** refers to the operational units and their interconnections that realize the architectural specifications. Examples of architectural attributes include the instruction set, the number of bits used to represent various data types (e.g., numbers, characters), I/O mechanisms, and techniques for addressing memory. Organizational attributes include those hardware details transparent to the programmer, such as control signals; interfaces between the computer and peripherals; and the memory technology used.

For example, it is an architectural design issue whether a computer will have a multiply instruction. It is an organizational issue whether that instruction will be implemented by a special multiply unit or by a mechanism that makes repeated use of the add unit of the system. The organizational decision may be based on the anticipated frequency of use of the multiply instruction, the relative speed of the two approaches, and the cost and physical size of a special multiply unit.

Historically, and still today, the distinction between architecture and organization has been an important one. Many computer manufacturers offer a family of computer models, all with the same architecture but with differences in organization. Consequently, the different models in the family have different price and performance characteristics. Furthermore, a particular architecture may span many years and encompass a number of different computer models, its organization changing with changing technology. A prominent example of both these phenomena is the IBM System/370 architecture. This architecture was first introduced in 1970 and

included a number of models. The customer with modest requirements could buy a cheaper, slower model and, if demand increased, later upgrade to a more expensive, faster model without having to abandon software that had already been developed. Over the years, IBM has introduced many new models with improved technology to replace older models, offering the customer greater speed, lower cost, or both. These newer models retained the same architecture so that the customer's software investment was protected. Remarkably, the System/370 architecture, with a few enhancements, has survived to this day as the architecture of IBM's mainframe product line.

In a class of computers called microcomputers, the relationship between architecture and organization is very close. Changes in technology not only influence organization but also result in the introduction of more powerful and more complex architectures. Generally, there is less of a requirement for generation-to-generation compatibility for these smaller machines. Thus, there is more interplay between organizational and architectural design decisions. An intriguing example of this is the reduced instruction set computer (RISC), which we examine in Chapter 15.

This book examines both computer organization and computer architecture. The emphasis is perhaps more on the side of organization. However, because a computer organization must be designed to implement a particular architectural specification, a thorough treatment of organization requires a detailed examination of architecture as well.

1.2 STRUCTURE AND FUNCTION

A computer is a complex system; contemporary computers contain millions of elementary electronic components. How, then, can one clearly describe them? The key is to recognize the hierarchical nature of most complex systems, including the computer [SIMO96]. A hierarchical system is a set of interrelated subsystems, each of the latter, in turn, hierarchical in structure until we reach some lowest level of elementary subsystem.

The hierarchical nature of complex systems is essential to both their design and their description. The designer need only deal with a particular level of the system at a time. At each level, the system consists of a set of components and their interrelationships. The behavior at each level depends only on a simplified, abstracted characterization of the system at the next lower level. At each level, the designer is concerned with structure and function:

- **Structure:** The way in which the components are interrelated.
- **Function:** The operation of each individual component as part of the structure.

In terms of description, we have two choices: starting at the bottom and building up to a complete description, or beginning with a top view and decomposing the system into its subparts. Evidence from a number of fields suggests that the top-down approach is the clearest and most effective [WEIN75].

The approach taken in this book follows from this viewpoint. The computer system will be described from the top down. We begin with the major components of a computer, describing their structure and function, and proceed to successively

lower layers of the hierarchy. The remainder of this section provides a very brief overview of this plan of attack.

Function

Both the structure and functioning of a computer are, in essence, simple. In general terms, there are only four basic functions that a computer can perform:

- **Data processing:** Data may take a wide variety of forms, and the range of processing requirements is broad. However, we shall see that there are only a few fundamental methods or types of data processing.
- **Data storage:** Even if the computer is processing data on the fly (i.e., data come in and get processed, and the results go out immediately), the computer must temporarily store at least those pieces of data that are being worked on at any given moment. Thus, there is at least a short-term data storage function. Equally important, the computer performs a long-term data storage function. Files of data are stored on the computer for subsequent retrieval and update.
- **Data movement:** The computer's operating environment consists of devices that serve as either sources or destinations of data. When data are received from or delivered to a device that is directly connected to the computer, the process is known as *input–output* (*I*/*O*), and the device is referred to as a *peripheral*. When data are moved over longer distances, to or from a remote device, the process is known as *data communications*.
- **Control:** Within the computer, a control unit manages the computer's resources and orchestrates the performance of its functional parts in response to instructions.

The preceding discussion may seem absurdly generalized. It is certainly possible, even at a top level of computer structure, to differentiate a variety of functions, but to quote [SIEW82]:

> There is remarkably little shaping of computer structure to fit the function to be performed. At the root of this lies the general-purpose nature of computers, in which all the functional specialization occurs at the time of programming and not at the time of design.

Structure

We now look in a general way at the internal structure of a computer. We begin with a traditional computer with a single processor that employs a microprogrammed control unit, then examine a typical multicore structure.

SIMPLE SINGLE-PROCESSOR COMPUTER Figure 1.1 provides a hierarchical view of the internal structure of a traditional single-processor computer. There are four main structural components:

- **Central processing unit (CPU):** Controls the operation of the computer and performs its data processing functions; often simply referred to as **processor**.
- **Main memory:** Stores data.

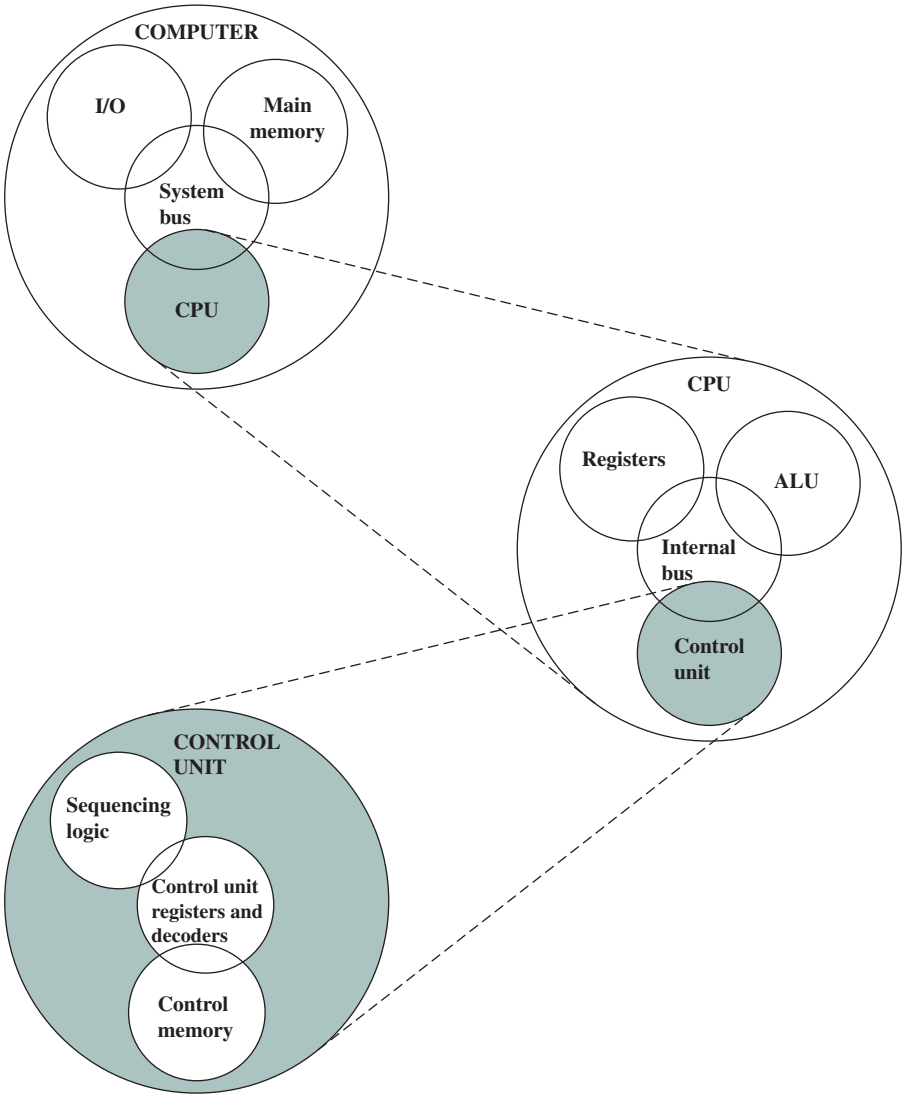


Figure 1.1 The Computer: Top-Level Structure

- **I/O:** Moves data between the computer and its external environment.
- **System interconnection:** Some mechanism that provides for communication among CPU, main memory, and I/O. A common example of system interconnection is by means of a **system bus**, consisting of a number of conducting wires to which all the other components attach.

There may be one or more of each of the aforementioned components. Traditionally, there has been just a single processor. In recent years, there has been increasing use of multiple processors in a single computer. Some design issues relating to multiple processors crop up and are discussed as the text proceeds; Part Five focuses on such computers.

Each of these components will be examined in some detail in Part Two. However, for our purposes, the most interesting and in some ways the most complex component is the CPU. Its major structural components are as follows:

- **Control unit:** Controls the operation of the CPU and hence the computer.
- **Arithmetic and logic unit (ALU):** Performs the computer's data processing functions.
- **Registers:** Provides storage internal to the CPU.
- **CPU interconnection:** Some mechanism that provides for communication among the control unit, ALU, and registers.

Part Three covers these components, where we will see that complexity is added by the use of parallel and pipelined organizational techniques. Finally, there are several approaches to the implementation of the control unit; one common approach is a *microprogrammed* implementation. In essence, a microprogrammed control unit operates by executing microinstructions that define the functionality of the control unit. With this approach, the structure of the control unit can be depicted, as in Figure 1.1. This structure is examined in Part Four.

MULTICORE COMPUTER STRUCTURE As was mentioned, contemporary computers generally have multiple processors. When these processors all reside on a single chip, the term *multicore computer* is used, and each processing unit (consisting of a control unit, ALU, registers, and perhaps cache) is called a *core*. To clarify the terminology, this text will use the following definitions.

- **Central processing unit (CPU):** That portion of a computer that fetches and executes instructions. It consists of an ALU, a control unit, and registers. In a system with a single processing unit, it is often simply referred to as a *processor*.
- **Core:** An individual processing unit on a processor chip. A core may be equivalent in functionality to a CPU on a single-CPU system. Other specialized processing units, such as one optimized for vector and matrix operations, are also referred to as cores.
- **Processor:** A physical piece of silicon containing one or more cores. The processor is the computer component that interprets and executes instructions. If a processor contains multiple cores, it is referred to as a **multicore processor**.

After about a decade of discussion, there is broad industry consensus on this usage.

Another prominent feature of contemporary computers is the use of multiple layers of memory, called *cache memory*, between the processor and main memory. Chapter 4 is devoted to the topic of cache memory. For our purposes in this section, we simply note that a cache memory is smaller and faster than main memory and is used to speed up memory access, by placing in the cache data from main memory, that is likely to be used in the near future. A greater performance improvement may be obtained by using multiple levels of cache, with level 1 (L1) closest to the core and additional levels (L2, L3, and so on) progressively farther from the core. In this scheme, level n is smaller and faster than level $n + 1$.

Figure 1.2 is a simplified view of the principal components of a typical multicore computer. Most computers, including embedded computers in smartphones and tablets, plus personal computers, laptops, and workstations, are housed on a motherboard. Before describing this arrangement, we need to define some terms. A **printed circuit board (PCB)** is a rigid, flat board that holds and interconnects chips and other electronic components. The board is made of layers, typically two to ten, that interconnect components via copper pathways that are etched into the board. The main printed circuit board in a computer is called a system board or **motherboard**, while smaller ones that plug into the slots in the main board are called expansion boards.

The most prominent elements on the motherboard are the chips. A **chip** is a single piece of semiconducting material, typically silicon, upon which electronic circuits and logic gates are fabricated. The resulting product is referred to as an **integrated circuit**.

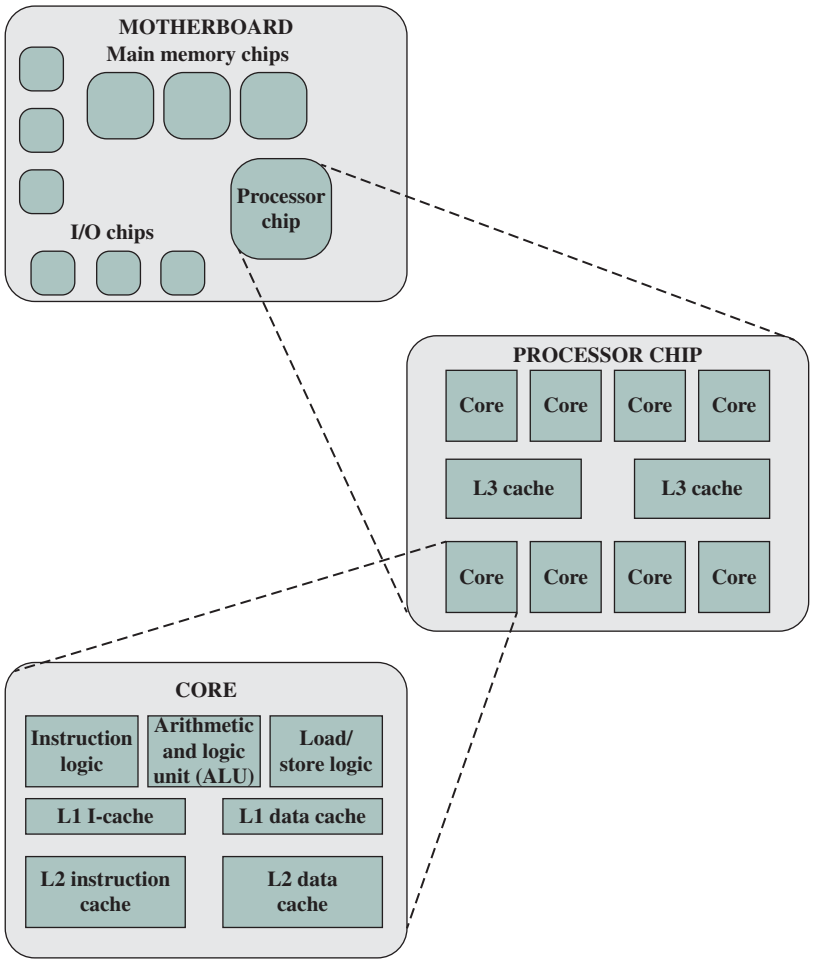


Figure 1.2 Simplified View of Major Elements of a Multicore Computer

The motherboard contains a slot or socket for the processor chip, which typically contains multiple individual cores, in what is known as a *multicore processor*. There are also slots for memory chips, I/O controller chips, and other key computer components. For desktop computers, expansion slots enable the inclusion of more components on expansion boards. Thus, a modern motherboard connects only a few individual chip components, with each chip containing from a few thousand up to hundreds of millions of transistors.

Figure 1.2 shows a processor chip that contains eight cores and an L3 cache. Not shown is the logic required to control operations between the cores and the cache and between the cores and the external circuitry on the motherboard. The figure indicates that the L3 cache occupies two distinct portions of the chip surface. However, typically, all cores have access to the entire L3 cache via the aforementioned control circuits. The processor chip shown in Figure 1.2 does not represent any specific product, but provides a general idea of how such chips are laid out.

Next, we zoom in on the structure of a single core, which occupies a portion of the processor chip. In general terms, the functional elements of a core are:

- **Instruction logic:** This includes the tasks involved in fetching instructions, and decoding each instruction to determine the instruction operation and the memory locations of any operands.
- **Arithmetic and logic unit (ALU):** Performs the operation specified by an instruction.
- **Load/store logic:** Manages the transfer of data to and from main memory via cache.

The core also contains an L1 cache, split between an instruction cache (I-cache) that is used for the transfer of instructions to and from main memory, and an L1 data cache, for the transfer of operands and results. Typically, today's processor chips also include an L2 cache as part of the core. In many cases, this cache is also split between instruction and data caches, although a combined, single L2 cache is also used.

Keep in mind that this representation of the layout of the core is only intended to give a general idea of internal core structure. In a given product, the functional elements may not be laid out as the three distinct elements shown in Figure 1.2, especially if some or all of these functions are implemented as part of a microprogrammed control unit.

EXAMPLES It will be instructive to look at some real-world examples that illustrate the hierarchical structure of computers. Figure 1.3 is a photograph of the motherboard for a computer built around two Intel Quad-Core Xeon processor chips. Many of the elements labeled on the photograph are discussed subsequently in this book. Here, we mention the most important, in addition to the processor sockets:

- PCI-Express slots for a high-end display adapter and for additional peripherals (Section 3.6 describes PCIe).
- Ethernet controller and Ethernet ports for network connections.
- USB sockets for peripheral devices.

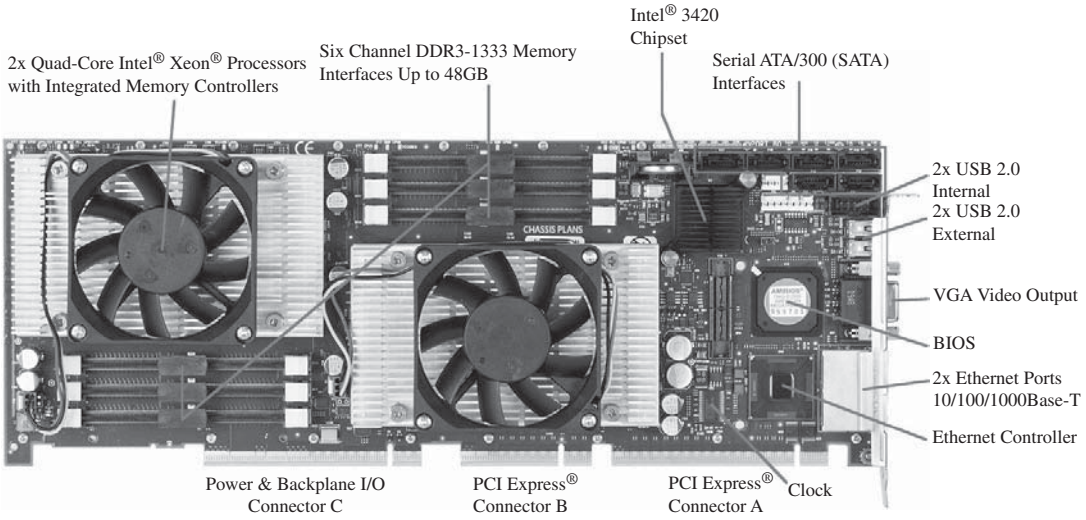


Figure 1.3 Motherboard with Two Intel Quad-Core Xeon Processors
Source: Chassis Plans, www.chassis-plans.com

- Serial ATA (SATA) sockets for connection to disk memory (Section 7.7 discusses Ethernet, USB, and SATA).
- Interfaces for DDR (double data rate) main memory chips (Section 5.3 discusses DDR).
- Intel 3420 chipset is an I/O controller for direct memory access operations between peripheral devices and main memory (Section 7.5 discusses DDR).

Following our top-down strategy, as illustrated in Figures 1.1 and 1.2, we can now zoom in and look at the internal structure of a processor chip. For variety, we look at an IBM chip instead of the Intel processor chip. Figure 1.4 is a photograph of the processor chip for the IBM zEnterprise EC12 mainframe computer. This chip has 2.75 billion transistors. The superimposed labels indicate how the silicon real estate of the chip is allocated. We see that this chip has six cores, or processors. In addition, there are two large areas labeled L3 cache, which are shared by all six processors. The L3 control logic controls traffic between the L3 cache and the cores and between the L3 cache and the external environment. Additionally, there is storage control (SC) logic between the cores and the L3 cache. The memory controller (MC) function controls access to memory external to the chip. The GX I/O bus controls the interface to the channel adapters accessing the I/O.

Going down one level deeper, we examine the internal structure of a single core, as shown in the photograph of Figure 1.5. Keep in mind that this is a portion of the silicon surface area making up a single-processor chip. The main sub-areas within this core area are the following:

- **ISU (instruction sequence unit):** Determines the sequence in which instructions are executed in what is referred to as a superscalar architecture (Chapter 16).
- **IFU (instruction fetch unit):** Logic for fetching instructions.

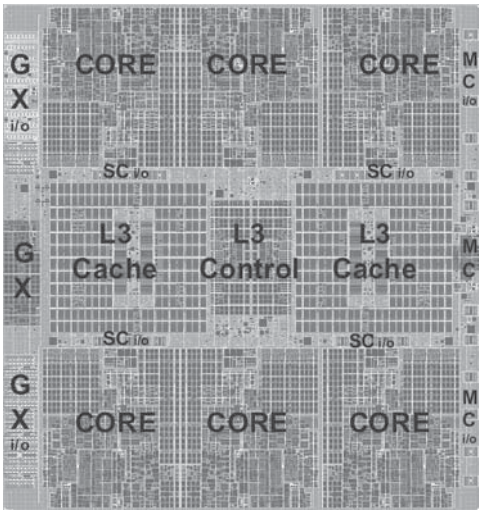


Figure 1.4 zEnterprise EC12 Processor Unit (PU) chip diagram
Source: IBM zEnterprise EC12 Technical Guide, December 2013, SG24-8049-01. IBM, Reprinted by Permission

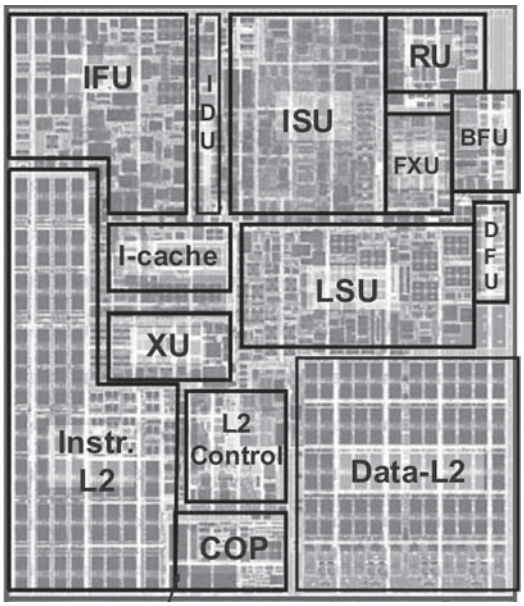


Figure 1.5 zEnterprise EC12 Core layout
Source: IBM zEnterprise EC12 Technical Guide, December 2013, SG24-8049-01. IBM, Reprinted by Permission

- **IDU (instruction decode unit):** The IDU is fed from the IFU buffers, and is responsible for the parsing and decoding of all z/Architecture operation codes.
- **LSU (load-store unit):** The LSU contains the 96-kB L1 data cache,[1] and manages data traffic between the L2 data cache and the functional execution units. It is responsible for handling all types of operand accesses of all lengths, modes, and formats as defined in the z/Architecture.
- **XU (translation unit):** This unit translates logical addresses from instructions into physical addresses in main memory. The XU also contains a translation lookaside buffer (TLB) used to speed up memory access. TLBs are discussed in Chapter 8.
- **FXU (fixed-point unit):** The FXU executes fixed-point arithmetic operations.
- **BFU (binary floating-point unit):** The BFU handles all binary and hexadecimal floating-point operations, as well as fixed-point multiplication operations.
- **DFU (decimal floating-point unit):** The DFU handles both fixed-point and floating-point operations on numbers that are stored as decimal digits.
- **RU (recovery unit):** The RU keeps a copy of the complete state of the system that includes all registers, collects hardware fault signals, and manages the hardware recovery actions.

[1]kB = kilobyte = 2048 bytes. Numerical prefixes are explained in a document under the "Other Useful" tab at ComputerScienceStudent.com.

- **COP (dedicated co-processor):** The COP is responsible for data compression and encryption functions for each core.
- **I-cache:** This is a 64-kB L1 instruction cache, allowing the IFU to prefetch instructions before they are needed.
- **L2 control:** This is the control logic that manages the traffic through the two L2 caches.
- **Data-L2:** A 1-MB L2 data cache for all memory traffic other than instructions.
- **Instr-L2:** A 1-MB L2 instruction cache.

As we progress through the book, the concepts introduced in this section will become clearer.

1.3 A BRIEF HISTORY OF COMPUTERS[2]

In this section, we provide a brief overview of the history of the development of computers. This history is interesting in itself, but more importantly, provides a basic introduction to many important concepts that we deal with throughout the book.

The First Generation: Vacuum Tubes

The first generation of computers used vacuum tubes for digital logic elements and memory. A number of research and then commercial computers were built using vacuum tubes. For our purposes, it will be instructive to examine perhaps the most famous first-generation computer, known as the IAS computer.

A fundamental design approach first implemented in the IAS computer is known as the *stored-program concept*. This idea is usually attributed to the mathematician John von Neumann. Alan Turing developed the idea at about the same time. The first publication of the idea was in a 1945 proposal by von Neumann for a new computer, the EDVAC (Electronic Discrete Variable Computer).[3]

In 1946, von Neumann and his colleagues began the design of a new stored-program computer, referred to as the IAS computer, at the Princeton Institute for Advanced Studies. The IAS computer, although not completed until 1952, is the prototype of all subsequent general-purpose computers.[4]

Figure 1.6 shows the structure of the IAS computer (compare with Figure 1.1). It consists of

- A **main memory**, which stores both data and instructions[5]
- An **arithmetic and logic unit (ALU)** capable of operating on binary data

---

[2]This book's Companion Web site (WilliamStallings.com/ComputerOrganization) contains several links to sites that provide photographs of many of the devices and components discussed in this section.

[3]The 1945 report on EDVAC is available at box.com/COA10e.

[4]A 1954 report [GOLD54] describes the implemented IAS machine and lists the final instruction set. It is available at box.com/COA10e.

[5]In this book, unless otherwise noted, the term *instruction* refers to a machine instruction that is directly interpreted and executed by the processor, in contrast to a statement in a high-level language, such as Ada or C++, which must first be compiled into a series of machine instructions before being executed.

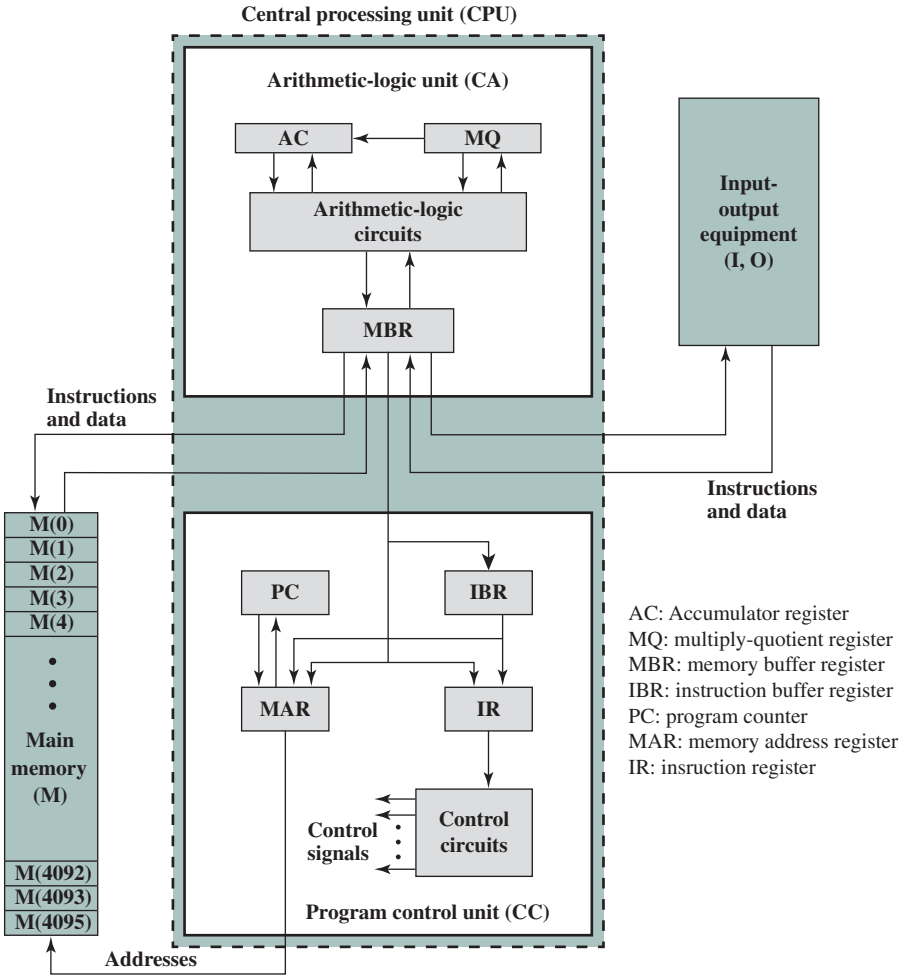


Figure 1.6 IAS Structure

- A **control unit**, which interprets the instructions in memory and causes them to be executed
- **Input–output (I/O)** equipment operated by the control unit

This structure was outlined in von Neumann's earlier proposal, which is worth quoting in part at this point [VONN45]:

> 2.2 **First:** Since the device is primarily a computer, it will have to perform the elementary operations of arithmetic most frequently. These are addition, subtraction, multiplication, and division. It is therefore reasonable that it should contain specialized organs for just these operations.

It must be observed, however, that while this principle as such is probably sound, the specific way in which it is realized requires close scrutiny. At any rate a *central arithmetical* part of the device will probably have to exist, and this constitutes *the first specific part: CA*.

2.3 **Second:** The logical control of the device, that is, the proper sequencing of its operations, can be most efficiently carried out by a central control organ. If the device is to be *elastic*, that is, as nearly as possible *all purpose*, then a distinction must be made between the specific instructions given for and defining a particular problem, and the general control organs that see to it that these instructions—no matter what they are—are carried out. The former must be stored in some way; the latter are represented by definite operating parts of the device. By the *central control* we mean this latter function only, and the organs that perform it form *the second specific part: CC*.

2.4 **Third:** Any device that is to carry out long and complicated sequences of operations (specifically of calculations) must have a considerable memory . . .

The instructions which govern a complicated problem may constitute considerable material, particularly so if the code is circumstantial (which it is in most arrangements). This material must be remembered.

At any rate, the total *memory* constitutes *the third specific part of the device: M*.

2.6 The three specific parts CA, CC (together C), and M correspond to the *associative* neurons in the human nervous system. It remains to discuss the equivalents of the *sensory* or *afferent* and the *motor* or *efferent* neurons. These are the *input* and *output* organs of the device.

The device must be endowed with the ability to maintain input and output (sensory and motor) contact with some specific medium of this type. The medium will be called the *outside recording medium of the device: R*.

2.7 **Fourth:** The device must have organs to transfer information from R into its specific parts C and M. These organs form its *input*, the *fourth specific part: I*. It will be seen that it is best to make all transfers from R (by I) into M and never directly from C.

2.8 **Fifth:** The device must have organs to transfer from its specific parts C and M into R. These organs form its *output*, *the fifth specific part: O*. It will be seen that it is again best to make all transfers from M (by O) into R, and never directly from C.

With rare exceptions, all of today's computers have this same general structure and function and are thus referred to as *von Neumann machines*. Thus, it is worthwhile at this point to describe briefly the operation of the IAS computer [BURK46, GOLD54]. Following [HAYE98], the terminology and notation of von Neumann

are changed in the following to conform more closely to modern usage; the examples accompanying this discussion are based on that latter text.

The memory of the IAS consists of 4,096 storage locations, called *words*, of 40 binary digits (bits) each.[6] Both data and instructions are stored there. Numbers are represented in binary form, and each instruction is a binary code. Figure 1.7 illustrates these formats. Each number is represented by a sign bit and a 39-bit value. A word may alternatively contain two 20-bit instructions, with each instruction consisting of an 8-bit operation code (opcode) specifying the operation to be performed and a 12-bit address designating one of the words in memory (numbered from 0 to 999).

The control unit operates the IAS by fetching instructions from memory and executing them one at a time. We explain these operations with reference to Figure 1.6. This figure reveals that both the control unit and the ALU contain storage locations, called *registers*, defined as follows:

- **Memory buffer register (MBR):** Contains a word to be stored in memory or sent to the I/O unit, or is used to receive a word from memory or from the I/O unit.
- **Memory address register (MAR):** Specifies the address in memory of the word to be written from or read into the MBR.
- **Instruction register (IR):** Contains the 8-bit opcode instruction being executed.
- **Instruction buffer register (IBR):** Employed to hold temporarily the right-hand instruction from a word in memory.
- **Program counter (PC):** Contains the address of the next instruction pair to be fetched from memory.
- **Accumulator (AC) and multiplier quotient (MQ):** Employed to hold temporarily operands and results of ALU operations. For example, the result

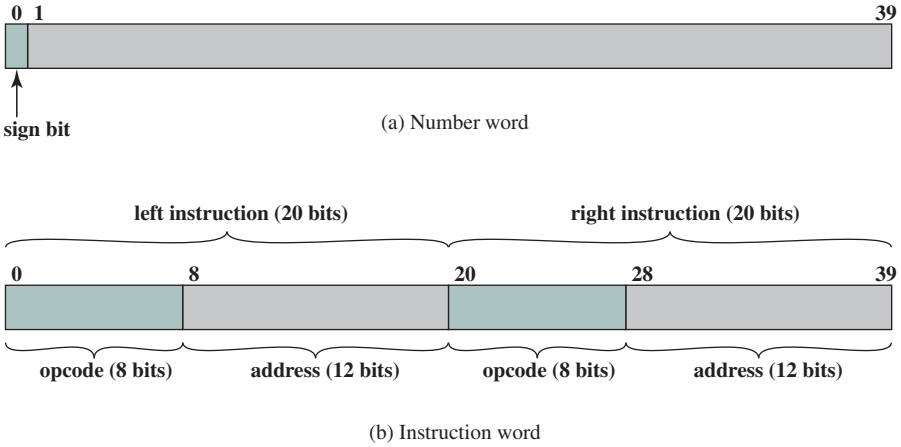


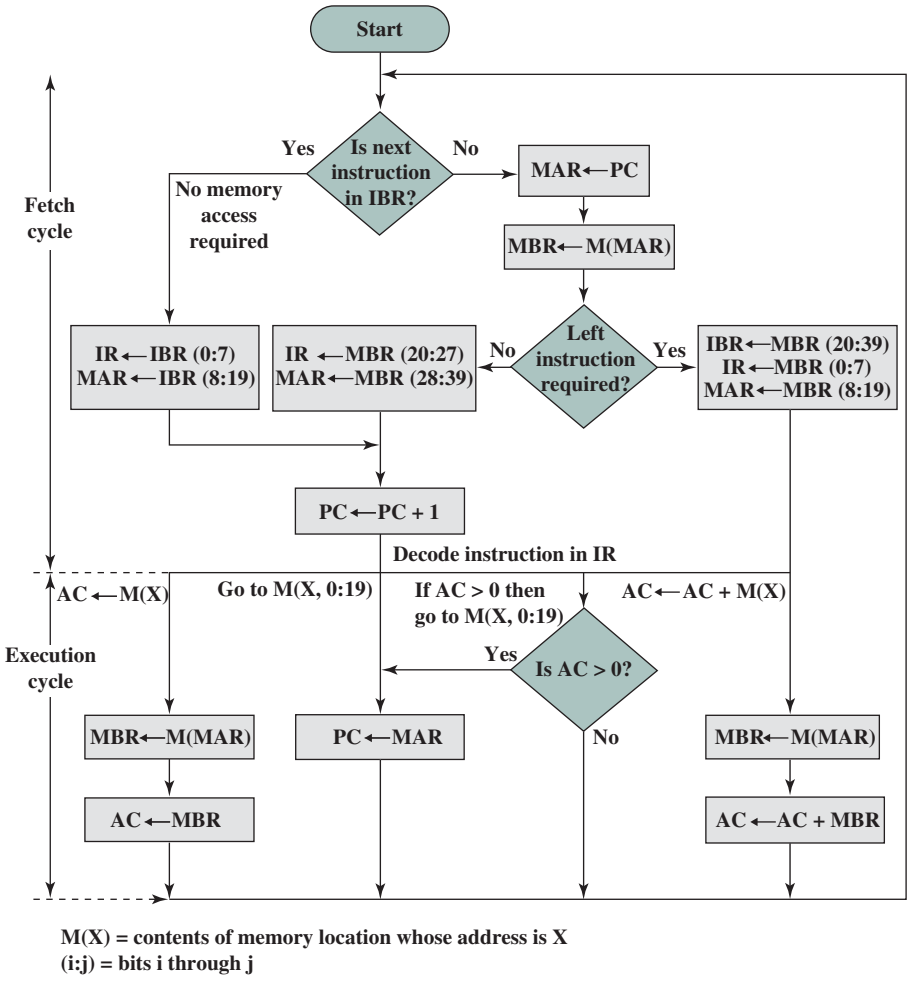
Figure 1.7 IAS Memory Formats

[6]There is no universal definition of the term *word*. In general, a word is an ordered set of bytes or bits that is the normal unit in which information may be stored, transmitted, or operated on within a given computer. Typically, if a processor has a fixed-length instruction set, then the instruction length equals the word length.

of multiplying two 40-bit numbers is an 80-bit number; the most significant 40 bits are stored in the AC and the least significant in the MQ.

The IAS operates by repetitively performing an *instruction cycle*, as shown in Figure 1.8. Each instruction cycle consists of two subcycles. During the *fetch cycle*, the opcode of the next instruction is loaded into the IR and the address portion is loaded into the MAR. This instruction may be taken from the IBR, or it can be obtained from memory by loading a word into the MBR, and then down to the IBR, IR, and MAR.

Why the indirection? These operations are controlled by electronic circuitry and result in the use of data paths. To simplify the electronics, there is only one register that is used to specify the address in memory for a read or write and only one register used for the source or destination.



M(X) = contents of memory location whose address is X
(i:j) = bits i through j

Figure 1.8 Partial Flowchart of IAS Operation

Once the opcode is in the IR, the *execute cycle* is performed. Control circuitry interprets the opcode and executes the instruction by sending out the appropriate control signals to cause data to be moved or an operation to be performed by the ALU.

The IAS computer had a total of 21 instructions, which are listed in Table 1.1. These can be grouped as follows:

- **Data transfer:** Move data between memory and ALU registers or between two ALU registers.
- **Unconditional branch:** Normally, the control unit executes instructions in sequence from memory. This sequence can be changed by a branch instruction, which facilitates repetitive operations.

Table 1.1 The IAS Instruction Set

Instruction Type	Opcode	Symbolic Representation	Description
Data transfer	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD –M(X)	Transfer –M(X) to the accumulator
	00000011	LOAD |M(X)|	Transfer absolute value of M(X) to the accumulator
	00000100	LOAD –|M(X)|	Transfer –|M(X)| to the accumulator
Unconditional branch	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
Conditional branch	00001111	JUMP + M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
	00010000	JUMP + M(X,20:39)	If number in the accumulator is nonnegative, take next instruction from right half of M(X)
Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD |M(X)|	Add |M(X)| to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB |M(X)|	Subtract |M(X)| from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2; that is, shift left one bit position
	00010101	RSH	Divide accumulator by 2; that is, shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 rightmost bits of AC

- **Conditional branch:** The branch can be made dependent on a condition, thus allowing decision points.
- **Arithmetic:** Operations performed by the ALU.
- **Address modify:** Permits addresses to be computed in the ALU and then inserted into instructions stored in memory. This allows a program considerable addressing flexibility.

Table 1.1 presents instructions (excluding I/O instructions) in a symbolic, easy-to-read form. In binary form, each instruction must conform to the format of Figure 1.7b. The opcode portion (first 8 bits) specifies which of the 21 instructions is to be executed. The address portion (remaining 12 bits) specifies which of the 4,096 memory locations is to be involved in the execution of the instruction.

Figure 1.8 shows several examples of instruction execution by the control unit. Note that each operation requires several steps, some of which are quite elaborate. The multiplication operation requires 39 suboperations, one for each bit position except that of the sign bit.

The Second Generation: Transistors

The first major change in the electronic computer came with the replacement of the vacuum tube by the transistor. The transistor, which is smaller, cheaper, and generates less heat than a vacuum tube, can be used in the same way as a vacuum tube to construct computers. Unlike the vacuum tube, which requires wires, metal plates, a glass capsule, and a vacuum, the transistor is a *solid-state device*, made from silicon.

The transistor was invented at Bell Labs in 1947 and by the 1950s had launched an electronic revolution. It was not until the late 1950s, however, that fully transistorized computers were commercially available. The use of the transistor defines the *second generation* of computers. It has become widely accepted to classify computers into generations based on the fundamental hardware technology employed (Table 1.2). Each new generation is characterized by greater processing performance, larger memory capacity, and smaller size than the previous one.

But there are other changes as well. The second generation saw the introduction of more complex arithmetic and logic units and control units, the use of high-level programming languages, and the provision of *system software* with the

Table 1.2 Computer Generations

Generation	Approximate Dates	Technology	Typical Speed (operations per second)
1	1946–1957	Vacuum tube	40,000
2	1957–1964	Transistor	200,000
3	1965–1971	Small- and medium-scale integration	1,000,000
4	1972–1977	Large scale integration	10,000,000
5	1978–1991	Very large scale integration	100,000,000
6	1991–	Ultra large scale integration	>1,000,000,000

computer. In broad terms, system software provided the ability to load programs, move data to peripherals, and libraries to perform common computations, similar to what modern operating systems, such as Windows and Linux, do.

It will be useful to examine an important member of the second generation: the IBM 7094 [BELL71]. From the introduction of the 700 series in 1952 to the introduction of the last member of the 7000 series in 1964, this IBM product line underwent an evolution that is typical of computer products. Successive members of the product line showed increased performance, increased capacity, and/or lower cost.

The size of main memory, in multiples of 2^{10} 36-bit words, grew from 2k ($1k = 2^{10}$) to 32k words,[7] while the time to access one word of memory, the *memory cycle time*, fell from 30 μs to 1.4 μs. The number of opcodes grew from a modest 24 to 185.

Also, over the lifetime of this series of computers, the relative speed of the CPU increased by a factor of 50. Speed improvements are achieved by improved electronics (e.g., a transistor implementation is faster than a vacuum tube implementation) and more complex circuitry. For example, the IBM 7094 includes an Instruction Backup Register, used to buffer the next instruction. The control unit fetches two adjacent words from memory for an instruction fetch. Except for the occurrence of a branching instruction, which is relatively infrequent (perhaps 10 to 15%), this means that the control unit has to access memory for an instruction on only half the instruction cycles. This prefetching significantly reduces the average instruction cycle time.

Figure 1.9 shows a large (many peripherals) configuration for an IBM 7094, which is representative of second-generation computers. Several differences from the IAS computer are worth noting. The most important of these is the use of *data channels*. A data channel is an independent I/O module with its own processor and instruction set. In a computer system with such devices, the CPU does not execute detailed I/O instructions. Such instructions are stored in a main memory to be executed by a special-purpose processor in the data channel itself. The CPU initiates an I/O transfer by sending a control signal to the data channel, instructing it to execute a sequence of instructions in memory. The data channel performs its task independently of the CPU and signals the CPU when the operation is complete. This arrangement relieves the CPU of a considerable processing burden.

Another new feature is the *multiplexor*, which is the central termination point for data channels, the CPU, and memory. The multiplexor schedules access to the memory from the CPU and data channels, allowing these devices to act independently.

The Third Generation: Integrated Circuits

A single, self-contained transistor is called a *discrete component*. Throughout the 1950s and early 1960s, electronic equipment was composed largely of discrete components—transistors, resistors, capacitors, and so on. Discrete components were manufactured separately, packaged in their own containers, and soldered or wired

[7] A discussion of the uses of numerical prefixes, such as kilo and giga, is contained in a supporting document at the Computer Science Student Resource Site at ComputerScienceStudent.com.

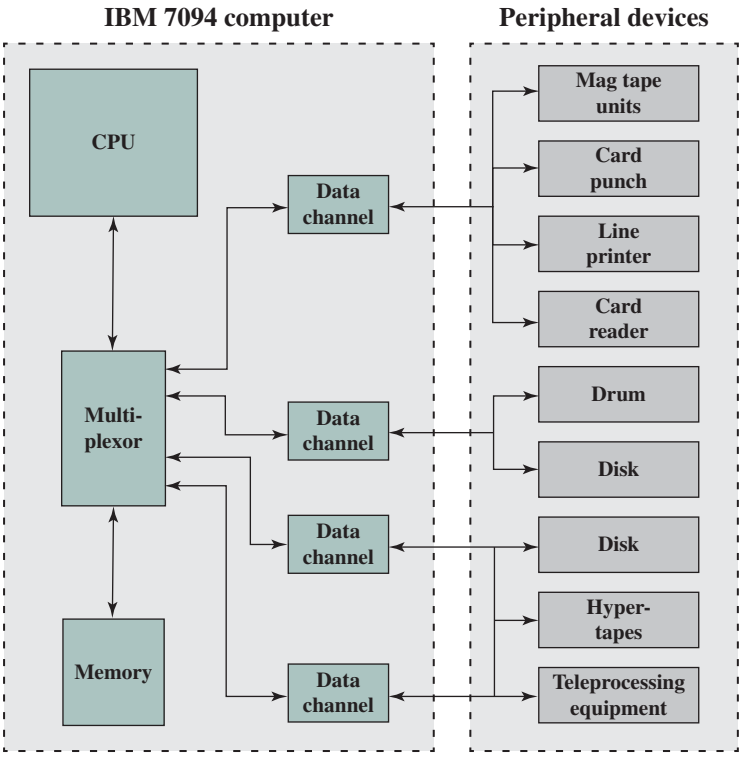


Figure 1.9 An IBM 7094 Configuration

together onto Masonite-like circuit boards, which were then installed in computers, oscilloscopes, and other electronic equipment. Whenever an electronic device called for a transistor, a little tube of metal containing a pinhead-sized piece of silicon had to be soldered to a circuit board. The entire manufacturing process, from transistor to circuit board, was expensive and cumbersome.

These facts of life were beginning to create problems in the computer industry. Early second-generation computers contained about 10,000 transistors. This figure grew to the hundreds of thousands, making the manufacture of newer, more powerful machines increasingly difficult.

In 1958 came the achievement that revolutionized electronics and started the era of microelectronics: the invention of the integrated circuit. It is the integrated circuit that defines the third generation of computers. In this section, we provide a brief introduction to the technology of integrated circuits. Then we look at perhaps the two most important members of the third generation, both of which were introduced at the beginning of that era: the IBM System/360 and the DEC PDP-8.

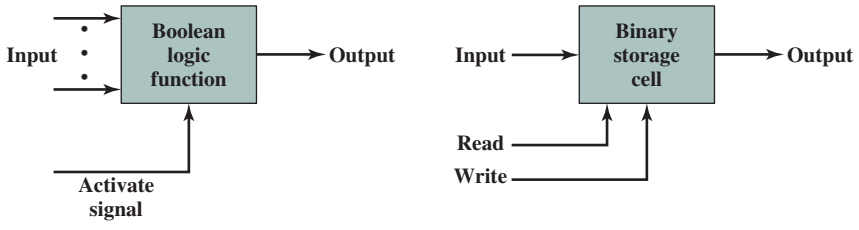
MICROELECTRONICS Microelectronics means, literally, "small electronics." Since the beginnings of digital electronics and the computer industry, there has been a persistent and consistent trend toward the reduction in size of digital electronic circuits. Before examining the implications and benefits of this trend, we need to say something about the nature of digital electronics. A more detailed discussion is found in Chapter 11.

The basic elements of a digital computer, as we know, must perform data storage, movement, processing, and control functions. Only two fundamental types of components are required (Figure 1.10): gates and memory cells. A **gate** is a device that implements a simple Boolean or logical function. For example, an AND gate with inputs *A* and *B* and output *C* implements the expression IF *A* AND *B* ARE TRUE THEN *C* IS TRUE. Such devices are called gates because they control data flow in much the same way that canal gates control the flow of water. The **memory cell** is a device that can store 1 bit of data; that is, the device can be in one of two stable states at any time. By interconnecting large numbers of these fundamental devices, we can construct a computer. We can relate this to our four basic functions as follows:

- **Data storage:** Provided by memory cells.
- **Data processing:** Provided by gates.
- **Data movement:** The paths among components are used to move data from memory to memory and from memory through gates to memory.
- **Control:** The paths among components can carry control signals. For example, a gate will have one or two data inputs plus a control signal input that activates the gate. When the control signal is ON, the gate performs its function on the data inputs and produces a data output. Conversely, when the control signal is OFF, the output line is null, such as the one produced by a high impedance state. Similarly, the memory cell will store the bit that is on its input lead when the WRITE control signal is ON and will place the bit that is in the cell on its output lead when the READ control signal is ON.

Thus, a computer consists of gates, memory cells, and interconnections among these elements. The gates and memory cells are, in turn, constructed of simple electronic components, such as transistors and capacitors.

The integrated circuit exploits the fact that such components as transistors, resistors, and conductors can be fabricated from a semiconductor such as silicon. It is merely an extension of the solid-state art to fabricate an entire circuit in a tiny piece of silicon rather than assemble discrete components made from separate pieces of silicon into the same circuit. Many transistors can be produced at the same time on a single wafer of silicon. Equally important, these transistors can be connected with a process of metallization to form circuits.



(a) Gate (b) Memory cell

Figure 1.10 Fundamental Computer Elements

Figure 1.11 depicts the key concepts in an integrated circuit. A thin *wafer* of silicon is divided into a matrix of small areas, each a few millimeters square. The identical circuit pattern is fabricated in each area, and the wafer is broken up into *chips*. Each chip consists of many gates and/or memory cells plus a number of input and output attachment points. This chip is then packaged in housing that protects it and provides pins for attachment to devices beyond the chip. A number of these packages can then be interconnected on a printed circuit board to produce larger and more complex circuits.

Initially, only a few gates or memory cells could be reliably manufactured and packaged together. These early integrated circuits are referred to as **small-scale integration (SSI)**. As time went on, it became possible to pack more and more components on the same chip. This growth in density is illustrated in Figure 1.12; it is one of the most remarkable technological trends ever recorded.[8] This figure reflects the famous Moore's law, which was propounded by Gordon Moore, cofounder of Intel, in 1965 [MOOR65]. Moore observed that the number of transistors that could be put on a single chip was doubling every year, and correctly predicted that this pace would continue into the near future. To the surprise of many, including Moore, the pace continued year after year and decade after decade. The pace slowed to a doubling every 18 months in the 1970s but has sustained that rate ever since.

The consequences of Moore's law are profound:

1. The cost of a chip has remained virtually unchanged during this period of rapid growth in density. This means that the cost of computer logic and memory circuitry has fallen at a dramatic rate.

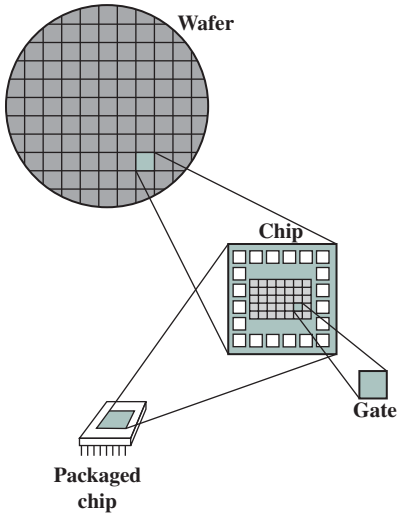


Figure 1.11 Relationship among Wafer, Chip, and Gate

[8]Note that the vertical axis uses a log scale. A basic review of log scales is in the math refresher document at the Computer Science Student Resource Site at ComputerScienceStudent.com.

2. Because logic and memory elements are placed closer together on more densely packed chips, the electrical path length is shortened, increasing operating speed.
3. The computer becomes smaller, making it more convenient to place in a variety of environments.
4. There is a reduction in power requirements.
5. The interconnections on the integrated circuit are much more reliable than solder connections. With more circuitry on each chip, there are fewer interchip connections.

IBM SYSTEM/360 By 1964, IBM had a firm grip on the computer market with its 7000 series of machines. In that year, IBM announced the System/360, a new family of computer products. Although the announcement itself was no surprise, it contained some unpleasant news for current IBM customers: the 360 product line was incompatible with older IBM machines. Thus, the transition to the 360 would be difficult for the current customer base, but IBM felt this was necessary to break out of some of the constraints of the 7000 architecture and to produce a system capable of evolving with the new integrated circuit technology [PADE81, GIFF87]. The strategy paid off both financially and technically. The 360 was the success of the decade and cemented IBM as the overwhelmingly dominant computer vendor, with a market share above 70%. And, with some modifications and extensions, the architecture of the 360 remains to this day the architecture of IBM's mainframe[9] computers. Examples using this architecture can be found throughout this text.

The System/360 was the industry's first planned family of computers. The family covered a wide range of performance and cost. The models were compatible in the

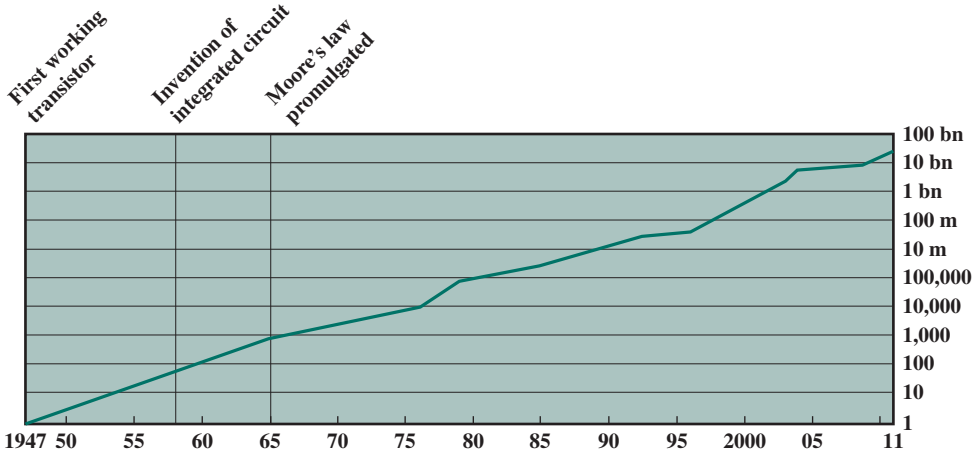


Figure 1.12 Growth in Transistor Count on Integrated Circuits

[9]The term *mainframe* is used for the larger, most powerful computers other than supercomputers. Typical characteristics of a mainframe are that it supports a large database, has elaborate I/O hardware, and is used in a central data processing facility.

sense that a program written for one model should be capable of being executed by another model in the series, with only a difference in the time it takes to execute.

The concept of a family of compatible computers was both novel and extremely successful. A customer with modest requirements and a budget to match could start with the relatively inexpensive Model 30. Later, if the customer's needs grew, it was possible to upgrade to a faster machine with more memory without sacrificing the investment in already-developed software. The characteristics of a family are as follows:

- **Similar or identical instruction set:** In many cases, the exact same set of machine instructions is supported on all members of the family. Thus, a program that executes on one machine will also execute on any other. In some cases, the lower end of the family has an instruction set that is a subset of that of the top end of the family. This means that programs can move up but not down.
- **Similar or identical operating system:** The same basic operating system is available for all family members. In some cases, additional features are added to the higher-end members.
- **Increasing speed:** The rate of instruction execution increases in going from lower to higher family members.
- **Increasing number of I/O ports:** The number of I/O ports increases in going from lower to higher family members.
- **Increasing memory size:** The size of main memory increases in going from lower to higher family members.
- **Increasing cost:** At a given point in time, the cost of a system increases in going from lower to higher family members.

How could such a family concept be implemented? Differences were achieved based on three factors: basic speed, size, and degree of simultaneity [STEV64]. For example, greater speed in the execution of a given instruction could be gained by the use of more complex circuitry in the ALU, allowing suboperations to be carried out in parallel. Another way of increasing speed was to increase the width of the data path between main memory and the CPU. On the Model 30, only 1 byte (8 bits) could be fetched from main memory at a time, whereas 8 bytes could be fetched at a time on the Model 75.

The System/360 not only dictated the future course of IBM but also had a profound impact on the entire industry. Many of its features have become standard on other large computers.

DEC PDP-8 In the same year that IBM shipped its first System/360, another momentous first shipment occurred: PDP-8 from Digital Equipment Corporation (DEC). At a time when the average computer required an air-conditioned room, the PDP-8 (dubbed a minicomputer by the industry, after the miniskirt of the day) was small enough that it could be placed on top of a lab bench or be built into other equipment. It could not do everything the mainframe could, but at $16,000, it was cheap enough for each lab technician to have one. In contrast, the System/360 series of mainframe computers introduced just a few months before cost hundreds of thousands of dollars.

The low cost and small size of the PDP-8 enabled another manufacturer to purchase a PDP-8 and integrate it into a total system for resale. These other manufacturers came to be known as **original equipment manufacturers (OEMs)**, and the OEM market became and remains a major segment of the computer marketplace.

In contrast to the central-switched architecture (Figure 1.9) used by IBM on its 700/7000 and 360 systems, later models of the PDP-8 used a structure that became virtually universal for microcomputers: the bus structure. This is illustrated in Figure 1.13. The PDP-8 bus, called the Omnibus, consists of 96 separate signal paths, used to carry control, address, and data signals. Because all system components share a common set of signal paths, their use can be controlled by the CPU. This architecture is highly flexible, allowing modules to be plugged into the bus to create various configurations. It is only in recent years that the bus structure has given way to a structure known as point-to-point interconnect, described in Chapter 3.

Later Generations

Beyond the third generation there is less general agreement on defining generations of computers. Table 1.2 suggests that there have been a number of later generations, based on advances in integrated circuit technology. With the introduction of **large-scale integration (LSI)**, more than 1,000 components can be placed on a single integrated circuit chip. Very-large-scale integration (VLSI) achieved more than 10,000 components per chip, while current ultra-large-scale integration (ULSI) chips can contain more than one billion components.

With the rapid pace of technology, the high rate of introduction of new products, and the importance of software and communications as well as hardware, the classification by generation becomes less clear and less meaningful. In this section, we mention two of the most important of developments in later generations.

SEMICONDUCTOR MEMORY The first application of integrated circuit technology to computers was the construction of the processor (the control unit and the arithmetic and logic unit) out of integrated circuit chips. But it was also found that this same technology could be used to construct memories.

In the 1950s and 1960s, most computer memory was constructed from tiny rings of ferromagnetic material, each about a sixteenth of an inch in diameter. These rings were strung up on grids of fine wires suspended on small screens inside the computer. Magnetized one way, a ring (called a *core*) represented a one; magnetized the other way, it stood for a zero. Magnetic-core memory was rather fast; it took as little as a millionth of a second to read a bit stored in memory. But it was

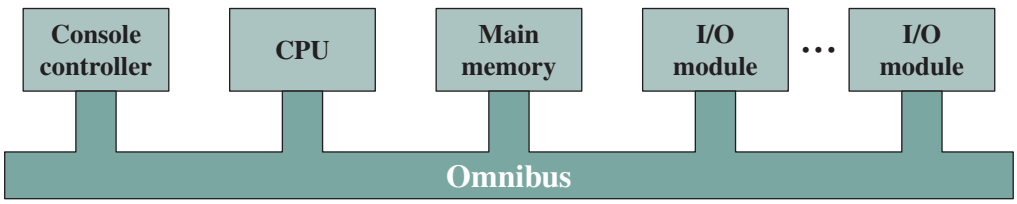


Figure 1.13 PDP-8 Bus Structure

expensive and bulky, and used destructive readout: The simple act of reading a core erased the data stored in it. It was therefore necessary to install circuits to restore the data as soon as it had been extracted.

Then, in 1970, Fairchild produced the first relatively capacious semiconductor memory. This chip, about the size of a single core, could hold 256 bits of memory. It was nondestructive and much faster than core. It took only 70 billionths of a second to read a bit. However, the cost per bit was higher than for that of core.

In 1974, a seminal event occurred: The price per bit of semiconductor memory dropped below the price per bit of core memory. Following this, there has been a continuing and rapid decline in memory cost accompanied by a corresponding increase in physical memory density. This has led the way to smaller, faster machines with memory sizes of larger and more expensive machines from just a few years earlier. Developments in memory technology, together with developments in processor technology to be discussed next, changed the nature of computers in less than a decade. Although bulky, expensive computers remain a part of the landscape, the computer has also been brought out to the "end user," with office machines and personal computers.

Since 1970, semiconductor memory has been through 13 generations: 1k, 4k, 16k, 64k, 256k, 1M, 4M, 16M, 64M, 256M, 1G, 4G, and, as of this writing, 8 Gb on a single chip ($1\text{k} = 2^{10}, 1\text{M} = 2^{20}, 1\text{G} = 2^{30}$). Each generation has provided increased storage density, accompanied by declining cost per bit and declining access time. Densities are projected to reach 16 Gb by 2018 and 32 Gb by 2023 [ITRS14].

MICROPROCESSORS Just as the density of elements on memory chips has continued to rise, so has the density of elements on processor chips. As time went on, more and more elements were placed on each chip, so that fewer and fewer chips were needed to construct a single computer processor.

A breakthrough was achieved in 1971, when Intel developed its 4004. The 4004 was the first chip to contain *all* of the components of a CPU on a single chip: The microprocessor was born.

The 4004 can add two 4-bit numbers and can multiply only by repeated addition. By today's standards, the 4004 is hopelessly primitive, but it marked the beginning of a continuing evolution of microprocessor capability and power.

This evolution can be seen most easily in the number of bits that the processor deals with at a time. There is no clear-cut measure of this, but perhaps the best measure is the data bus width: the number of bits of data that can be brought into or sent out of the processor at a time. Another measure is the number of bits in the accumulator or in the set of general-purpose registers. Often, these measures coincide, but not always. For example, a number of microprocessors were developed that operate on 16-bit numbers in registers but can only read and write 8 bits at a time.

The next major step in the evolution of the microprocessor was the introduction in 1972 of the Intel 8008. This was the first 8-bit microprocessor and was almost twice as complex as the 4004.

Neither of these steps was to have the impact of the next major event: the introduction in 1974 of the Intel 8080. This was the first general-purpose microprocessor. Whereas the 4004 and the 8008 had been designed for specific applications, the 8080 was designed to be the CPU of a general-purpose microcomputer. Like the

8008, the 8080 is an 8-bit microprocessor. The 8080, however, is faster, has a richer instruction set, and has a large addressing capability.

About the same time, 16-bit microprocessors began to be developed. However, it was not until the end of the 1970s that powerful, general-purpose 16-bit microprocessors appeared. One of these was the 8086. The next step in this trend occurred in 1981, when both Bell Labs and Hewlett-Packard developed 32-bit, single-chip microprocessors. Intel introduced its own 32-bit microprocessor, the 80386, in 1985 (Table 1.3).

Table 1.3 Evolution of Intel Microprocessors (page 1 of 2)

(a) 1970s Processors

	4004	8008	8080	8086	8088
Introduced	1971	1972	1974	1978	1979
Clock speeds	108 kHz	108 kHz	2 MHz	5 MHz, 8 MHz, 10 MHz	5 MHz, 8 MHz
Bus width	4 bits	8 bits	8 bits	16 bits	8 bits
Number of transistors	2,300	3,500	6,000	29,000	29,000
Feature size (μm)	10	8	6	3	6
Addressable memory	640 bytes	16 KB	64 KB	1 MB	1 MB

(b) 1980s Processors

	80286	386TM DX	386TM SX	486TM DX CPU
Introduced	1982	1985	1988	1989
Clock speeds	6–12.5 MHz	16–33 MHz	16–33 MHz	25–50 MHz
Bus width	16 bits	32 bits	16 bits	32 bits
Number of transistors	134,000	275,000	275,000	1.2 million
Feature size (μm)	1.5	1	1	0.8–1
Addressable memory	16 MB	4 GB	16 MB	4 GB
Virtual memory	1 GB	64 TB	64 TB	64 TB
Cache	—	—	—	8 kB

(c) 1990s Processors

	486TM SX	Pentium	Pentium Pro	Pentium II
Introduced	1991	1993	1995	1997
Clock speeds	16–33 MHz	60–166 MHz,	150–200 MHz	200–300 MHz
Bus width	32 bits	32 bits	64 bits	64 bits
Number of transistors	1.185 million	3.1 million	5.5 million	7.5 million
Feature size (μm)	1	0.8	0.6	0.35
Addressable memory	4 GB	4 GB	64 GB	64 GB
Virtual memory	64 TB	64 TB	64 TB	64 TB
Cache	8 kB	8 kB	512 kB L1 and 1 MB L2	512 kB L2

(d) Recent Processors

	Pentium III	Pentium 4	Core 2 Duo	Core i7 EE 4960X
Introduced	1999	2000	2006	2013
Clock speeds	450–660 MHz	1.3–1.8 GHz	1.06–1.2 GHz	4 GHz
Bus width	64 bits	64 bits	64 bits	64 bits
Number of transistors	9.5 million	42 million	167 million	1.86 billion
Feature size (nm)	250	180	65	22
Addressable memory	64 GB	64 GB	64 GB	64 GB
Virtual memory	64 TB	64 TB	64 TB	64 TB
Cache	512 kB L2	256 kB L2	2 MB L2	1.5 MB L2/15 MB L3
Number of cores	1	1	2	6

1.4 THE EVOLUTION OF THE INTEL x86 ARCHITECTURE

Throughout this book, we rely on many concrete examples of computer design and implementation to illustrate concepts and to illuminate trade-offs. Numerous systems, both contemporary and historical, provide examples of important computer architecture design features. But the book relies principally on examples from two processor families: the Intel x86 and the ARM architectures. The current x86 offerings represent the results of decades of design effort on **complex instruction set computers (CISCs)**. The x86 incorporates the sophisticated design principles once found only on mainframes and supercomputers and serves as an excellent example of CISC design. An alternative approach to processor design is the **reduced instruction set computer (RISC)**. The ARM architecture is used in a wide variety of embedded systems and is one of the most powerful and best-designed RISC-based systems on the market. In this section and the next, we provide a brief overview of these two systems.

In terms of market share, Intel has ranked as the number one maker of microprocessors for non-embedded systems for decades, a position it seems unlikely to yield. The evolution of its flagship microprocessor product serves as a good indicator of the evolution of computer technology in general.

Table 1.3 shows that evolution. Interestingly, as microprocessors have grown faster and much more complex, Intel has actually picked up the pace. Intel used to develop microprocessors one after another, every four years. But Intel hopes to keep rivals at bay by trimming a year or two off this development time, and has done so with the most recent x86 generations.[10]

[10]Intel refers to this as the *tick-tock model*. Using this model, Intel has successfully delivered next-generation silicon technology as well as new processor microarchitecture on alternating years for the past several years. See http://www.intel.com/content/www/us/en/silicon-innovations/intel-tick-tock-model-general.html.

It is worthwhile to list some of the highlights of the evolution of the Intel product line:

- **8080:** The world's first general-purpose microprocessor. This was an 8-bit machine, with an 8-bit data path to memory. The 8080 was used in the first personal computer, the Altair.
- **8086:** A far more powerful, 16-bit machine. In addition to a wider data path and larger registers, the 8086 sported an instruction cache, or queue, that prefetches a few instructions before they are executed. A variant of this processor, the 8088, was used in IBM's first personal computer, securing the success of Intel. The 8086 is the first appearance of the x86 architecture.
- **80286:** This extension of the 8086 enabled addressing a 16-MB memory instead of just 1 MB.
- **80386:** Intel's first 32-bit machine, and a major overhaul of the product. With a 32-bit architecture, the 80386 rivaled the complexity and power of minicomputers and mainframes introduced just a few years earlier. This was the first Intel processor to support multitasking, meaning it could run multiple programs at the same time.
- **80486:** The 80486 introduced the use of much more sophisticated and powerful cache technology and sophisticated instruction pipelining. The 80486 also offered a built-in math coprocessor, offloading complex math operations from the main CPU.
- **Pentium:** With the Pentium, Intel introduced the use of superscalar techniques, which allow multiple instructions to execute in parallel.
- **Pentium Pro:** The Pentium Pro continued the move into superscalar organization begun with the Pentium, with aggressive use of register renaming, branch prediction, data flow analysis, and speculative execution.
- **Pentium II:** The Pentium II incorporated Intel MMX technology, which is designed specifically to process video, audio, and graphics data efficiently.
- **Pentium III:** The Pentium III incorporates additional floating-point instructions: The Streaming SIMD Extensions (SSE) instruction set extension added 70 new instructions designed to increase performance when exactly the same operations are to be performed on multiple data objects. Typical applications are digital signal processing and graphics processing.
- **Pentium 4:** The Pentium 4 includes additional floating-point and other enhancements for multimedia.[11]
- **Core:** This is the first Intel x86 microprocessor with a dual core, referring to the implementation of two cores on a single chip.
- **Core 2:** The Core 2 extends the Core architecture to 64 bits. The Core 2 Quad provides four cores on a single chip. More recent Core offerings have up to 10 cores per chip. An important addition to the architecture was the Advanced Vector Extensions instruction set that provided a set of 256-bit, and then 512-bit, instructions for efficient processing of vector data.

[11]With the Pentium 4, Intel switched from Roman numerals to Arabic numerals for model numbers.

Almost 40 years after its introduction in 1978, the x86 architecture continues to dominate the processor market outside of embedded systems. Although the organization and technology of the x86 machines have changed dramatically over the decades, the instruction set architecture has evolved to remain backward compatible with earlier versions. Thus, any program written on an older version of the x86 architecture can execute on newer versions. All changes to the instruction set architecture have involved additions to the instruction set, with no subtractions. The rate of change has been the addition of roughly one instruction per month added to the architecture [ANTH08], so that there are now thousands of instructions in the instruction set.

The x86 provides an excellent illustration of the advances in computer hardware over the past 35 years. The 1978 8086 was introduced with a clock speed of 5 MHz and had 29,000 transistors. A six-core Core i7 EE 4960X introduced in 2013 operates at 4 GHz, a speedup of a factor of 800, and has 1.86 billion transistors, about 64,000 times as many as the 8086. Yet the Core i7 EE 4960X is in only a slightly larger package than the 8086 and has a comparable cost.

1.5 EMBEDDED SYSTEMS

The term *embedded system* refers to the use of electronics and software within a product, as opposed to a general-purpose computer, such as a laptop or desktop system. Millions of computers are sold every year, including laptops, personal computers, workstations, servers, mainframes, and supercomputers. In contrast, billions of computer systems are produced each year that are embedded within larger devices. Today, many, perhaps most, devices that use electric power have an embedded computing system. It is likely that in the near future virtually all such devices will have embedded computing systems.

Types of devices with embedded systems are almost too numerous to list. Examples include cell phones, digital cameras, video cameras, calculators, microwave ovens, home security systems, washing machines, lighting systems, thermostats, printers, various automotive systems (e.g., transmission control, cruise control, fuel injection, anti-lock brakes, and suspension systems), tennis rackets, toothbrushes, and numerous types of sensors and actuators in automated systems.

Often, embedded systems are tightly coupled to their environment. This can give rise to real-time constraints imposed by the need to interact with the environment. Constraints, such as required speeds of motion, required precision of measurement, and required time durations, dictate the timing of software operations. If multiple activities must be managed simultaneously, this imposes more complex real-time constraints.

Figure 1.14 shows in general terms an embedded system organization. In addition to the processor and memory, there are a number of elements that differ from the typical desktop or laptop computer:

- There may be a variety of interfaces that enable the system to measure, manipulate, and otherwise interact with the external environment. Embedded systems often interact (sense, manipulate, and communicate) with external world through sensors and actuators and hence are typically reactive systems; a

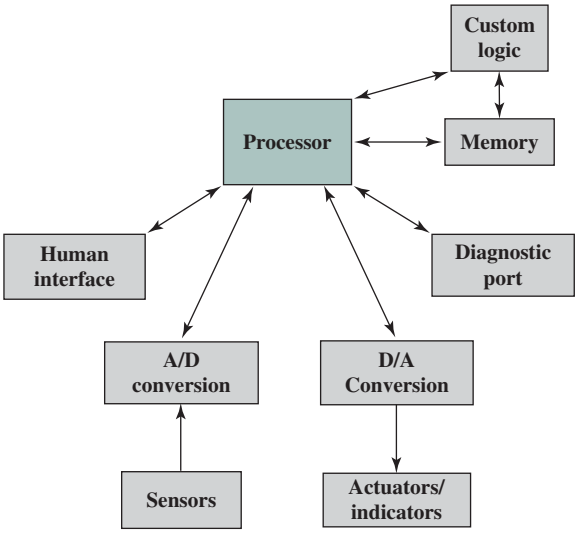


Figure 1.14 Possible Organization of an Embedded System

reactive system is in continual interaction with the environment and executes at a pace determined by that environment.

- The human interface may be as simple as a flashing light or as complicated as real-time robotic vision. In many cases, there is no human interface.
- The diagnostic port may be used for diagnosing the system that is being controlled—not just for diagnosing the computer.
- Special-purpose field programmable (FPGA), application-specific (ASIC), or even nondigital hardware may be used to increase performance or reliability.
- Software often has a fixed function and is specific to the application.
- Efficiency is of paramount importance for embedded systems. They are optimized for energy, code size, execution time, weight and dimensions, and cost.

There are several noteworthy areas of similarity to general-purpose computer systems as well:

- Even with nominally fixed function software, the ability to field upgrade to fix bugs, to improve security, and to add functionality, has become very important for embedded systems, and not just in consumer devices.
- One comparatively recent development has been of embedded system platforms that support a wide variety of apps. Good examples of this are smartphones and audio/visual devices, such as smart TVs.

The Internet of Things

It is worthwhile to separately callout one of the major drivers in the proliferation of embedded systems. The **Internet of things (IoT)** is a term that refers to the expanding

interconnection of smart devices, ranging from appliances to tiny sensors. A dominant theme is the embedding of short-range mobile transceivers into a wide array of gadgets and everyday items, enabling new forms of communication between people and things, and between things themselves. The Internet now supports the interconnection of billions of industrial and personal objects, usually through cloud systems. The objects deliver sensor information, act on their environment, and, in some cases, modify themselves, to create overall management of a larger system, like a factory or city.

The IoT is primarily driven by deeply embedded devices (defined below). These devices are low-bandwidth, low-repetition data-capture, and low-bandwidth data-usage appliances that communicate with each other and provide data via user interfaces. Embedded appliances, such as high-resolution video security cameras, video VoIP phones, and a handful of others, require high-bandwidth streaming capabilities. Yet countless products simply require packets of data to be intermittently delivered.

With reference to the end systems supported, the Internet has gone through roughly four generations of deployment culminating in the IoT:

1. **Information technology (IT):** PCs, servers, routers, firewalls, and so on, bought as IT devices by enterprise IT people and primarily using wired connectivity.
2. **Operational technology (OT):** Machines/appliances with embedded IT built by non-IT companies, such as medical machinery, SCADA (supervisory control and data acquisition), process control, and kiosks, bought as appliances by enterprise OT people and primarily using wired connectivity.
3. **Personal technology:** Smartphones, tablets, and eBook readers bought as IT devices by consumers (employees) exclusively using wireless connectivity and often multiple forms of wireless connectivity.
4. **Sensor/actuator technology:** Single-purpose devices bought by consumers, IT, and OT people exclusively using wireless connectivity, generally of a single form, as part of larger systems.

It is the fourth generation that is usually thought of as the IoT, and it is marked by the use of billions of embedded devices.

Embedded Operating Systems

There are two general approaches to developing an embedded operating system (OS). The first approach is to take an existing OS and adapt it for the embedded application. For example, there are embedded versions of Linux, Windows, and Mac, as well as other commercial and proprietary operating systems specialized for embedded systems. The other approach is to design and implement an OS intended solely for embedded use. An example of the latter is TinyOS, widely used in wireless sensor networks. This topic is explored in depth in [STAL15].

Application Processors versus Dedicated Processors

In this subsection, and the next two, we briefly introduce some terms commonly found in the literature on embedded systems. **Application processors** are defined

by the processor's ability to execute complex operating systems, such as Linux, Android, and Chrome. Thus, the application processor is general-purpose in nature. A good example of the use of an embedded application processor is the smartphone. The embedded system is designed to support numerous apps and perform a wide variety of functions.

Most embedded systems employ a **dedicated processor**, which, as the name implies, is dedicated to one or a small number of specific tasks required by the host device. Because such an embedded system is dedicated to a specific task or tasks, the processor and associated components can be engineered to reduce size and cost.

Microprocessors versus Microcontrollers

As we have seen, early **microprocessor** chips included registers, an ALU, and some sort of control unit or instruction processing logic. As transistor density increased, it became possible to increase the complexity of the instruction set architecture, and ultimately to add memory and more than one processor. Contemporary microprocessor chips, as shown in Figure 1.2, include multiple cores and a substantial amount of cache memory.

A **microcontroller** chip makes a substantially different use of the logic space available. Figure 1.15 shows in general terms the elements typically found on a microcontroller chip. As shown, a microcontroller is a single chip that contains the processor, non-volatile memory for the program (ROM), volatile memory for input and output (RAM), a clock, and an I/O control unit. The processor portion of the microcontroller has a much lower silicon area than other microprocessors and much higher energy efficiency. We examine microcontroller organization in more detail in Section 1.6.

Also called a "computer on a chip," billions of microcontroller units are embedded each year in myriad products from toys to appliances to automobiles. For example, a single vehicle can use 70 or more microcontrollers. Typically, especially for the smaller, less expensive microcontrollers, they are used as dedicated processors for specific tasks. For example, microcontrollers are heavily utilized in automation processes. By providing simple reactions to input, they can control machinery, turn fans on and off, open and close valves, and so forth. They are integral parts of modern industrial technology and are among the most inexpensive ways to produce machinery that can handle extremely complex functionalities.

Microcontrollers come in a range of physical sizes and processing power. Processors range from 4-bit to 32-bit architectures. Microcontrollers tend to be much slower than microprocessors, typically operating in the MHz range rather than the GHz speeds of microprocessors. Another typical feature of a microcontroller is that it does not provide for human interaction. The microcontroller is programmed for a specific task, embedded in its device, and executes as and when required.

Embedded versus Deeply Embedded Systems

We have, in this section, defined the concept of an embedded system. A subset of embedded systems, and a quite numerous subset, is referred to as **deeply embedded systems**. Although this term is widely used in the technical and commercial

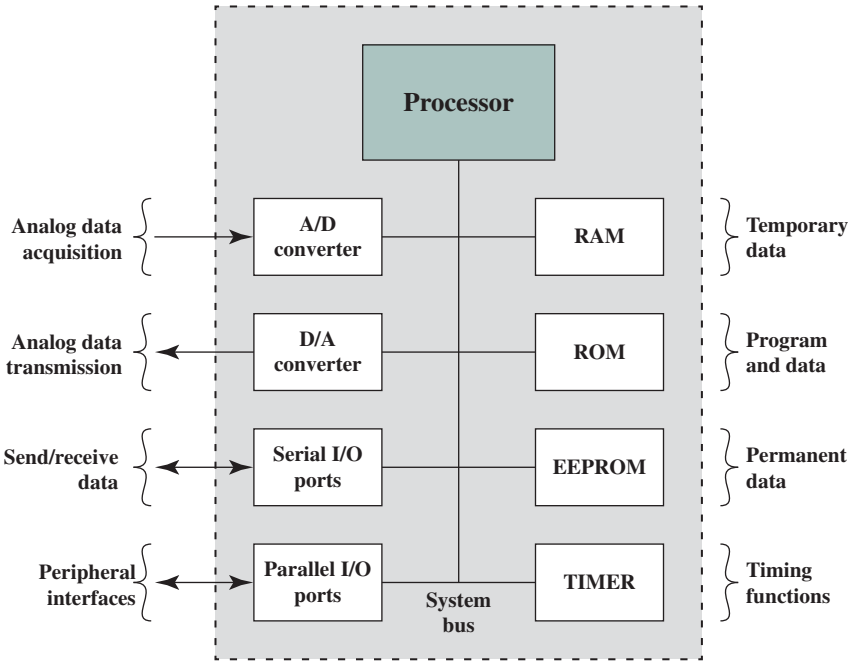


Figure 1.15 Typical Microcontroller Chip Elements

literature, you will search the Internet in vain (or at least I did) for a straightforward definition. Generally, we can say that a deeply embedded system has a processor whose behavior is difficult to observe both by the programmer and the user. A deeply embedded system uses a microcontroller rather than a microprocessor, is not programmable once the program logic for the device has been burned into ROM (read-only memory), and has no interaction with a user.

Deeply embedded systems are dedicated, single-purpose devices that detect something in the environment, perform a basic level of processing, and then do something with the results. Deeply embedded systems often have wireless capability and appear in networked configurations, such as networks of sensors deployed over a large area (e.g., factory, agricultural field). The Internet of things depends heavily on deeply embedded systems. Typically, deeply embedded systems have extreme resource constraints in terms of memory, processor size, time, and power consumption.

1.6 ARM ARCHITECTURE

The ARM architecture refers to a processor architecture that has evolved from RISC design principles and is used in embedded systems. Chapter 15 examines RISC design principles in detail. In this section, we give a brief overview of the ARM architecture.

ARM Evolution

ARM is a family of RISC-based microprocessors and microcontrollers designed by ARM Holdings, Cambridge, England. The company doesn't make processors but instead designs microprocessor and multicore architectures and licenses them to manufacturers. Specifically, ARM Holdings has two types of licensable products: processors and processor architectures. For processors, the customer buys the rights to use ARM-supplied design in their own chips. For a processor architecture, the customer buys the rights to design their own processor compliant with ARM's architecture.

ARM chips are high-speed processors that are known for their small die size and low power requirements. They are widely used in smartphones and other handheld devices, including game systems, as well as a large variety of consumer products. ARM chips are the processors in Apple's popular iPod and iPhone devices, and are used in virtually all Android smartphones as well. ARM is probably the most widely used embedded processor architecture and indeed the most widely used processor architecture of any kind in the world [VANC14].

The origins of ARM technology can be traced back to the British-based Acorn Computers company. In the early 1980s, Acorn was awarded a contract by the British Broadcasting Corporation (BBC) to develop a new microcomputer architecture for the BBC Computer Literacy Project. The success of this contract enabled Acorn to go on to develop the first commercial RISC processor, the Acorn RISC Machine (ARM). The first version, ARM1, became operational in 1985 and was used for internal research and development as well as being used as a coprocessor in the BBC machine.

In this early stage, Acorn used the company VLSI Technology to do the actual fabrication of the processor chips. VLSI was licensed to market the chip on its own and had some success in getting other companies to use the ARM in their products, particularly as an embedded processor.

The ARM design matched a growing commercial need for a high-performance, low-power-consumption, small-size, and low-cost processor for embedded applications. But further development was beyond the scope of Acorn's capabilities. Accordingly, a new company was organized, with Acorn, VLSI, and Apple Computer as founding partners, known as ARM Ltd. The Acorn RISC Machine became Advanced RISC Machines.[12]

Instruction Set Architecture

The ARM instruction set is highly regular, designed for efficient implementation of the processor and efficient execution. All instructions are 32 bits long and follow a regular format. This makes the ARM ISA suitable for implementation over a wide range of products.

Augmenting the basic ARM ISA is the Thumb instruction set, which is a re-encoded subset of the ARM instruction set. Thumb is designed to increase the performance of ARM implementations that use a 16-bit or narrower memory data bus,

[12]The company dropped the designation *Advanced RISC Machines* in the late 1990s. It is now simply known as the ARM architecture.

and to allow better code density than provided by the ARM instruction set. The Thumb instruction set contains a subset of the ARM 32-bit instruction set recoded into 16-bit instructions. The current defined version is Thumb-2.

The ARM and Thumb-2 ISAs are discussed in Chapters 12 and 13.

ARM Products

ARM Holdings licenses a number of specialized microprocessors and related technologies, but the bulk of their product line is the Cortex family of microprocessor architectures. There are three Cortex architectures, conveniently labeled with the initials A, R, and M.

CORTEX-A/CORTEX-A50 The Cortex-A and Cortex-A50 are application processors, intended for mobile devices such as smartphones and eBook readers, as well as consumer devices such as digital TV and home gateways (e.g., DSL and cable Internet modems). These processors run at higher clock frequency (over 1 GHz), and support a memory management unit (MMU), which is required for full feature OSs such as Linux, Android, MS Windows, and mobile OSs. An MMU is a hardware module that supports virtual memory and paging by translating virtual addresses into physical addresses; this topic is explored in Chapter 8.

The two architectures use both the ARM and Thumb-2 instruction sets; the principal difference is that the Cortex-A is a 32-bit machine, and the Cortex-A50 is a 64-bit machine.

CORTEX-R The Cortex-R is designed to support real-time applications, in which the timing of events needs to be controlled with rapid response to events. They can run at a fairly high clock frequency (e.g., 200MHz to 800MHz) and have very low response latency. The Cortex-R includes enhancements both to the instruction set and to the processor organization to support deeply embedded real-time devices. Most of these processors do not have MMU; the limited data requirements and the limited number of simultaneous processes eliminates the need for elaborate hardware and software support for virtual memory. The Cortex-R does have a Memory Protection Unit (MPU), cache, and other memory features designed for industrial applications. An MPU is a hardware module that prohibits one program in memory from accidentally accessing memory assigned to another active program. Using various methods, a protective boundary is created around the program, and instructions within the program are prohibited from referencing data outside of that boundary.

Examples of embedded systems that would use the Cortex-R are automotive braking systems, mass storage controllers, and networking and printing devices.

CORTEX-M Cortex-M series processors have been developed primarily for the microcontroller domain where the need for fast, highly deterministic interrupt management is coupled with the desire for extremely low gate count and lowest possible power consumption. As with the Cortex-R series, the Cortex-M architecture has an MPU but no MMU. The Cortex-M uses only the Thumb-2 instruction set. The market for the Cortex-M includes IoT devices, wireless sensor/actuator networks used in factories and other enterprises, automotive body electronics, and so on.

There are currently four versions of the Cortex-M series:

- **Cortex-M0:** Designed for 8- and 16-bit applications, this model emphasizes low cost, ultra low power, and simplicity. It is optimized for small silicon die size (starting from 12k gates) and use in the lowest cost chips.
- **Cortex-M0+:** An enhanced version of the M0 that is more energy efficient.
- **Cortex-M3:** Designed for 16- and 32-bit applications, this model emphasizes performance and energy efficiency. It also has comprehensive debug and trace features to enable software developers to develop their applications quickly.
- **Cortex-M4:** This model provides all the features of the Cortex-M3, with additional instructions to support digital signal processing tasks.

In this text, we will primarily use the ARM Cortex-M3 as our example embedded system processor. It is the best suited of all ARM models for general-purpose microcontroller use. The Cortex-M3 is used by a variety of manufacturers of microcontroller products. Initial microcontroller devices from lead partners already combine the Cortex-M3 processor with flash, SRAM, and multiple peripherals to provide a competitive offering at the price of just $1.

Figure 1.16 provides a block diagram of the EFM32 microcontroller from Silicon Labs. The figure also shows detail of the Cortex-M3 processor and core components. We examine each level in turn.

The **Cortex-M3 core** makes use of separate buses for instructions and data. This arrangement is sometimes referred to as a Harvard architecture, in contrast with the von Neumann architecture, which uses the same signal buses and memory for both instructions and data. By being able to read both an instruction and data from memory at the same time, the Cortex-M3 processor can perform many operations in parallel, speeding application execution. The core contains a decoder for Thumb instructions, an advanced ALU with support for hardware multiply and divide, control logic, and interfaces to the other components of the processor. In particular, there is an interface to the nested vector interrupt controller (NVIC) and the embedded trace macrocell (ETM) module.

The core is part of a module called the **Cortex-M3 processor**. This term is somewhat misleading, because typically in the literature, the terms core and processor are viewed as equivalent. In addition to the core, the processor includes the following elements:

- **NVIC:** Provides configurable interrupt handling abilities to the processor. It facilitates low-latency exception and interrupt handling, and controls power management.
- **ETM:** An optional debug component that enables reconstruction of program execution. The ETM is designed to be a high-speed, low-power debug tool that only supports instruction trace.
- **Debug access port (DAP):** This provides an interface for external debug access to the processor.
- **Debug logic:** Basic debug functionality includes processor halt, single-step, processor core register access, unlimited software breakpoints, and full system memory access.

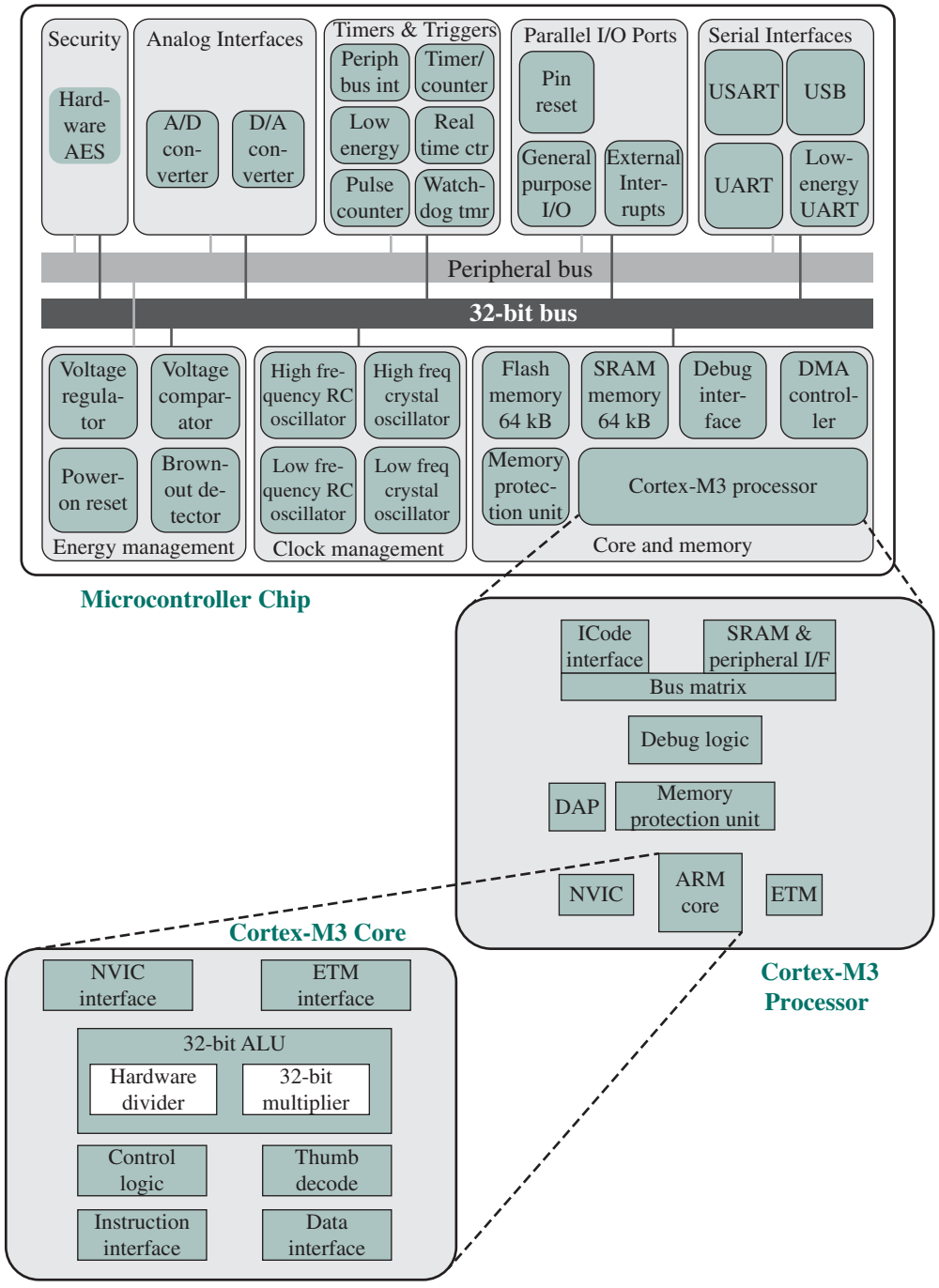


Figure 1.16 Typical Microcontroller Chip Based on Cortex-M3

- **ICode interface:** Fetches instructions from the code memory space.
- **SRAM & peripheral interface:** Read/write interface to data memory and peripheral devices.
- **Bus matrix:** Connects the core and debug interfaces to external buses on the microcontroller.
- **Memory protection unit:** Protects critical data used by the operating system from user applications, separating processing tasks by disallowing access to each other's data, disabling access to memory regions, allowing memory regions to be defined as read-only, and detecting unexpected memory accesses that could potentially break the system.

The upper part of Figure 1.16 shows the block diagram of a typical microcontroller built with the Cortex-M3, in this case the EFM32 microcontroller. This microcontroller is marketed for use in a wide variety of devices, including energy, gas, and water metering; alarm and security systems; industrial automation devices; home automation devices; smart accessories; and health and fitness devices. The silicon chip consists of 10 main areas:[13]

- **Core and memory:** This region includes the Cortex-M3 processor, static RAM (SRAM) data memory,[14] and flash memory[15] for storing program instructions and nonvarying application data. Flash memory is nonvolatile (data is not lost when power is shut off) and so is ideal for this purpose. The SRAM stores variable data. This area also includes a debug interface, which makes it easy to reprogram and update the system in the field.
- **Parallel I/O ports:** Configurable for a variety of parallel I/O schemes.
- **Serial interfaces:** Supports various serial I/O schemes.
- **Analog interfaces:** Analog-to-digital and digital-to-analog logic to support sensors and actuators.
- **Timers and triggers:** Keeps track of timing and counts events, generates output waveforms, and triggers timed actions in other peripherals.
- **Clock management:** Controls the clocks and oscillators on the chip. Multiple clocks and oscillators are used to minimize power consumption and provide short startup times.
- **Energy management:** Manages the various low-energy modes of operation of the processor and peripherals to provide real-time management of the energy needs so as to minimize energy consumption.
- **Security:** The chip includes a hardware implementation of the Advanced Encryption Standard (AES).

---

[13]This discussion does not go into details about all of the individual modules; for the interested reader, an in-depth discussion is provided in the document EFM32G200.pdf, available at box.com/COA10e.

[14]Static RAM (SRAM) is a form of random-access memory used for cache memory; see Chapter 5.

[15]Flash memory is a versatile form of memory used both in microcontrollers and as external memory; it is discussed in Chapter 6.

- **32-bit bus:** Connects all of the components on the chip.
- **Peripheral bus:** A network which lets the different peripheral module communicate directly with each other without involving the processor. This supports timing-critical operation and reduces software overhead.

Comparing Figure 1.16 with Figure 1.2, you will see many similarities and the same general hierarchical structure. Note, however, that the top level of a microcontroller computer system is a single chip, whereas for a multicore computer, the top level is a motherboard containing a number of chips. Another noteworthy difference is that there is no cache, neither in the Cortex-M3 processor nor in the microcontroller as a whole, which plays an important role if the code or data resides in external memory. Though the number of cycles to read the instruction or data varies depending on cache hit or miss, the cache greatly improves the performance when external memory is used. Such overhead is not needed for a microcontroller.

1.7 CLOUD COMPUTING

Although the general concepts for cloud computing go back to the 1950s, cloud computing services first became available in the early 2000s, particularly targeted at large enterprises. Since then, cloud computing has spread to small and medium size businesses, and most recently to consumers. Apple's iCloud was launched in 2012 and had 20 million users within a week of launch. Evernote, the cloud-based notetaking and archiving service, launched in 2008, approached 100 million users in less than 6 years. In this section, we provide a brief overview. Cloud computing is examined in more detail in Chapter 17.

Basic Concepts

There is an increasingly prominent trend in many organizations to move a substantial portion or even all information technology (IT) operations to an Internet-connected infrastructure known as enterprise cloud computing. At the same time, individual users of PCs and mobile devices are relying more and more on cloud computing services to backup data, synch devices, and share, using personal cloud computing. NIST defines cloud computing, in NIST SP-800-145 (*The NIST Definition of Cloud Computing*), as follows:

> **Cloud computing:** A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

Basically, with cloud computing, you get economies of scale, professional network management, and professional security management. These features can be attractive to companies large and small, government agencies, and individual PC and mobile users. The individual or company only needs to pay for the storage

capacity and services they need. The user, be it company or individual, doesn't have the hassle of setting up a database system, acquiring the hardware they need, doing maintenance, and backing up the data—all these are part of the cloud service.

In theory, another big advantage of using cloud computing to store your data and share it with others is that the cloud provider takes care of security. Alas, the customer is not always protected. There have been a number of security failures among cloud providers. Evernote made headlines in early 2013 when it told all of its users to reset their passwords after an intrusion was discovered.

Cloud networking refers to the networks and network management functionality that must be in place to enable cloud computing. Most cloud computing solutions rely on the Internet, but that is only a piece of the networking infrastructure. One example of cloud networking is the provisioning of high-performance and/or high-reliability networking between the provider and subscriber. In this case, some or all of the traffic between an enterprise and the cloud bypasses the Internet and uses dedicated private network facilities owned or leased by the cloud service provider. More generally, cloud networking refers to the collection of network capabilities required to access a cloud, including making use of specialized services over the Internet, linking enterprise data centers to a cloud, and using firewalls and other network security devices at critical points to enforce access security policies.

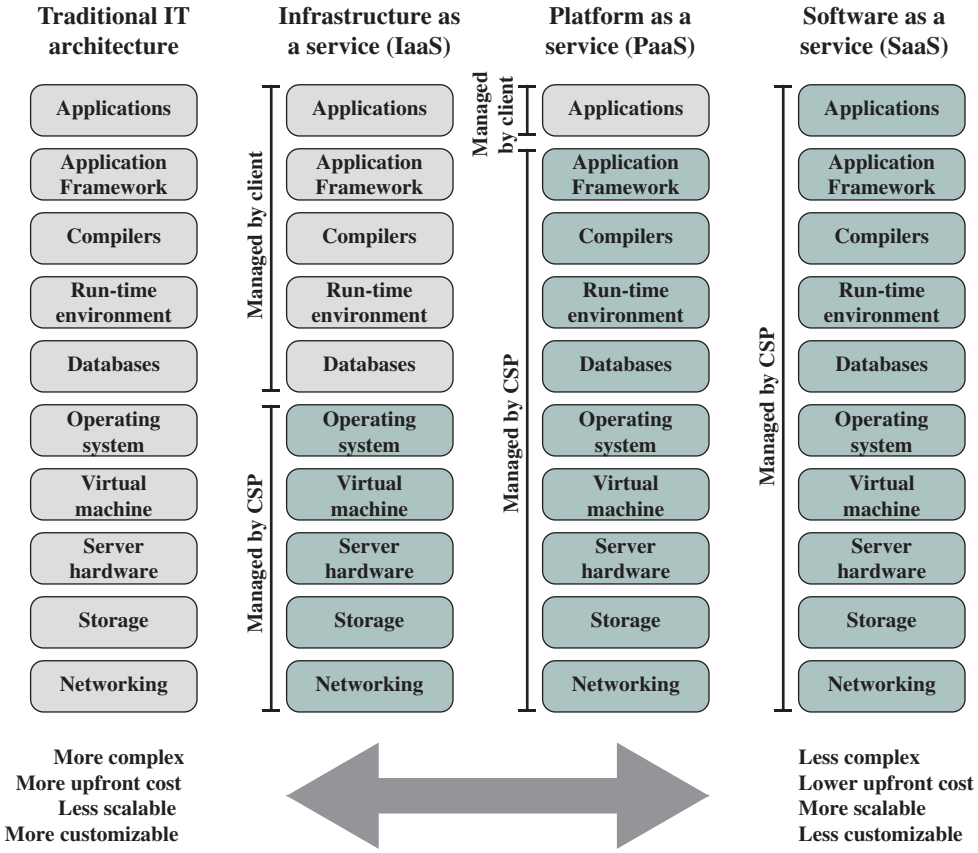
We can think of **cloud storage** as a subset of cloud computing. In essence, cloud storage consists of database storage and database applications hosted remotely on cloud servers. Cloud storage enables small businesses and individual users to take advantage of data storage that scales with their needs and to take advantage of a variety of database applications without having to buy, maintain, and manage the storage assets.

Cloud Services

The essential purpose of cloud computing is to provide for the convenient rental of computing resources. A cloud service provider (CSP) maintains computing and data storage resources that are available over the Internet or private networks. Customers can rent a portion of these resources as needed. Virtually all cloud service is provided using one of three models (Figure 1.17): SaaS, PaaS, and IaaS, which we examine in this section.

SOFTWARE AS A SERVICE (SAAS) As the name implies, a SaaS cloud provides service to customers in the form of software, specifically application software, running on and accessible in the cloud. SaaS follows the familiar model of Web services, in this case applied to cloud resources. SaaS enables the customer to use the cloud provider's applications running on the provider's cloud infrastructure. The applications are accessible from various client devices through a simple interface such as a Web browser. Instead of obtaining desktop and server licenses for software products it uses, an enterprise obtains the same functions from the cloud service. SaaS saves the complexity of software installation, maintenance, upgrades, and patches. Examples of services at this level are Gmail, Google's e-mail service, and Salesforce.com, which help firms keep track of their customers.

Common subscribers to SaaS are organizations that want to provide their employees with access to typical office productivity software, such as document



IT = information technology
CSP = cloud service provider

Figure 1.17 Alternative Information Technology Architectures

management and email. Individuals also commonly use the SaaS model to acquire cloud resources. Typically, subscribers use specific applications on demand. The cloud provider also usually offers data-related features such as automatic backup and data sharing between subscribers.

PLATFORM AS A SERVICE (PAAS) A PaaS cloud provides service to customers in the form of a platform on which the customer's applications can run. PaaS enables the customer to deploy onto the cloud infrastructure containing customer-created or acquired applications. A PaaS cloud provides useful software building blocks, plus a number of development tools, such as programming languages, run-time environments, and other tools that assist in deploying new applications. In effect, PaaS is an operating system in the cloud. PaaS is useful for an organization that wants to develop new or tailored applications while paying for the needed computing resources only as needed and only for as long as needed. Google App Engine and the Salesforce1 Platform from Salesforce.com are examples of PaaS.

INFRASTRUCTURE AS A SERVICE (IAAS) With IaaS, the customer has access to the underlying cloud infrastructure. IaaS provides virtual machines and other abstracted hardware and operating systems, which may be controlled through a service application programming interface (API). IaaS offers the customer processing, storage, networks, and other fundamental computing resources so that the customer is able to deploy and run arbitrary software, which can include operating systems and applications. IaaS enables customers to combine basic computing services, such as number crunching and data storage, to build highly adaptable computer systems. Examples of IaaS are Amazon Elastic Compute Cloud (Amazon EC2) and Windows Azure.

1.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

application processor	gate	microprocessor
arithmetic and logic unit (ALU)	infrastructure as a service (IaaS)	motherboard
ARM	input–output (I/O)	multicore
central processing unit (CPU)	instruction set architecture (ISA)	multicore processor
chip	integrated circuit	original equipment manufacturer (OEM)
cloud computing	Intel x86	platform as a service (PaaS)
cloud networking	Internet of things (IoT)	printed circuit board
cloud storage	main memory	processor
computer architecture	memory cell	registers
computer organization	memory management unit (MMU)	semiconductor
control unit	memory protection unit (MPU)	semiconductor memory
core	microcontroller	software as a service (SaaS)
dedicated processor	microelectronics	system bus
deeply embedded system		system interconnection
embedded system		vacuum tubes

Review Questions

1.1 What, in general terms, is the distinction between computer organization and computer architecture?

1.2 What, in general terms, is the distinction between computer structure and computer function?

1.3 What are the four main functions of a computer?

1.4 List and briefly define the main structural components of a computer.

1.5 List and briefly define the main structural components of a processor.

1.6 What is a stored program computer?

1.7 Explain Moore's law.

1.8 List and explain the key characteristics of a computer family.

1.9 What is the key distinguishing feature of a microprocessor?

Problems

1.1 You are to write an IAS program to compute the results of the following equation.

$$Y = \sum_{X=1}^{N} X$$

Assume that the computation does not result in an arithmetic overflow and that X, Y, and N are positive integers with $N \geq 1$. *Note*: The IAS did not have assembly language, only machine language.

a. Use the equation $\text{Sum}(Y) = \frac{N(N+1)}{2}$ when writing the IAS program.

b. Do it the "hard way," without using the equation from part (a).

1.2 **a.** On the IAS, what would the machine code instruction look like to load the contents of memory address 2 to the accumulator?

b. How many trips to memory does the CPU need to make to complete this instruction during the instruction cycle?

1.3 On the IAS, describe in English the process that the CPU must undertake to read a value from memory and to write a value to memory in terms of what is put into the MAR, MBR, address bus, data bus, and control bus.

1.4 Given the memory contents of the IAS computer shown below,

Address	Contents
08A	010FA210FB
08B	010FA0F08D
08C	020FA210FB

show the assembly language code for the program, starting at address 08A. Explain what this program does.

1.5 In Figure 1.6, indicate the width, in bits, of each data path (e.g., between AC and ALU).

1.6 In the IBM 360 Models 65 and 75, addresses are staggered in two separate main memory units (e.g., all even-numbered words in one unit and all odd-numbered words in another). What might be the purpose of this technique?

1.7 The relative performance of the IBM 360 Model 75 is 50 times that of the 360 Model 30, yet the instruction cycle time is only 5 times as fast. How do you account for this discrepancy?

1.8 While browsing at Billy Bob's computer store, you overhear a customer asking Billy Bob what is the fastest computer in the store that he can buy. Billy Bob replies, "You're looking at our Macintoshes. The fastest Mac we have runs at a clock speed of 1.2 GHz. If you really want the fastest machine, you should buy our 2.4-GHz Intel Pentium IV instead." Is Billy Bob correct? What would you say to help this customer?

1.9 The ENIAC, a precursor to the ISA machine, was a decimal machine, in which each register was represented by a ring of 10 vacuum tubes. At any time, only one vacuum tube was in the ON state, representing one of the 10 decimal digits. Assuming that ENIAC had the capability to have multiple vacuum tubes in the ON and OFF state simultaneously, why is this representation "wasteful" and what range of integer values could we represent using the 10 vacuum tubes?

1.10 For each of the following examples, determine whether this is an embedded system, explaining why or why not.

a. Are programs that understand physics and/or hardware embedded? For example, one that uses finite-element methods to predict fluid flow over airplane wings?

b. Is the internal microprocessor controlling a disk drive an example of an embedded system?

c. I/O drivers control hardware, so does the presence of an I/O driver imply that the computer executing the driver is embedded?
d. Is a PDA (Personal Digital Assistant) an embedded system?
e. Is the microprocessor controlling a cell phone an embedded system?
f. Are the computers in a big phased-array radar considered embedded? These radars are 10-story buildings with one to three 100-foot diameter radiating patches on the sloped sides of the building.
g. Is a traditional flight management system (FMS) built into an airplane cockpit considered embedded?
h. Are the computers in a hardware-in-the-loop (HIL) simulator embedded?
i. Is the computer controlling a pacemaker in a person's chest an embedded computer?
j. Is the computer controlling fuel injection in an automobile engine embedded?